

LangChain项目-点餐系统

LangChain项目资料

链接: <https://pan.baidu.com/s/1G8lTev4xMiEoXyiR2Ycf7g> 提取码: gtb5

一、项目介绍

1.1 项目说明

本项目使用 `langchain` 构建一个智能点餐 `Agent` , 该 `Agent` 为用户解决如下问题:

- 基于FAQ回答常见/静态问题;
- 基于用户口味进行菜品推荐;

1.2 项目结构

本项目代码结构如下:

```
1  |—— [ .venv ]                # 虚拟环境
2  |—— [ agent ]                # agent构建代码目录
3  |   |—— langchain_assistant.py  # 构建agent主代码文件
4  |   |—— milvus_data_sync.py    # milvus数据同步代码文件
5  |   |—— [ prompt ]
6  |   |   |—— system_prompt.txt
7  |   |—— redis_data_sync.py    # redis数据同步代码文件
8  |—— [ api ]                  # 定义api接口代码目录
9  |   |—— main.py               # 项目所有接口定义目录
10 |—— [ models ]                # 嵌入模型目录
11 |—— [ ui ]                    # 前端代码目录
12 |—— [ configs ]
13 |   |—— config.py             # 定义项目需要的配置常量
14 |—— .env                      # 项目依赖的环境变量
15 |—— .gitignore                # git忽略文件
16 |—— menu.sql
17 |—— pyproject.toml            # 项目元数据和依赖声明
18 |—— run.py                    # 后端服务执行入口
19 |—— uv.lock                   # uv虚拟环境依赖锁文件
```

1.3 前端启动入口

前端启动前, 需要保证在Windows当中已安装了Node.js运行环境。**npm 是 Node.js 的包管理器** - npm (Node Package Manager) 随 Node.js 一起安装。在课程资料目录下面, 已经提供了 `.msi` 安装文件。双击下载的 `.msi` 文件, 按照向导完成安装。

安装完成后, 通过如下命令能够正常查看版本号, 即安装完成。

```
1  node --version
2  npm --version
```

安装完成后, 进入到前端ui目录下面, 通过如下命令, 即可启动前端脚本:

```
1 start.bat
```

启动后，如下输出即表示启动成功：

```
1 =====
2 点餐系统 - 前端启动
3 =====
4
5 正在检查Node.js环境...
6 Node.js环境正常
7
8 正在检查依赖包...
9 依赖包已存在
10
11 正在启动开发服务器...
12 前端地址：http://localhost:3000
13 确保后端API运行在：http://localhost:8000
14
15 按 Ctrl+C 停止服务
16 =====
17
18 > aimenu-frontend@1.0.0 dev
19 > vite
```

1.4 后端启动入口

在项目根目录下面，通过以下命令启动：

```
1 python run.py
```

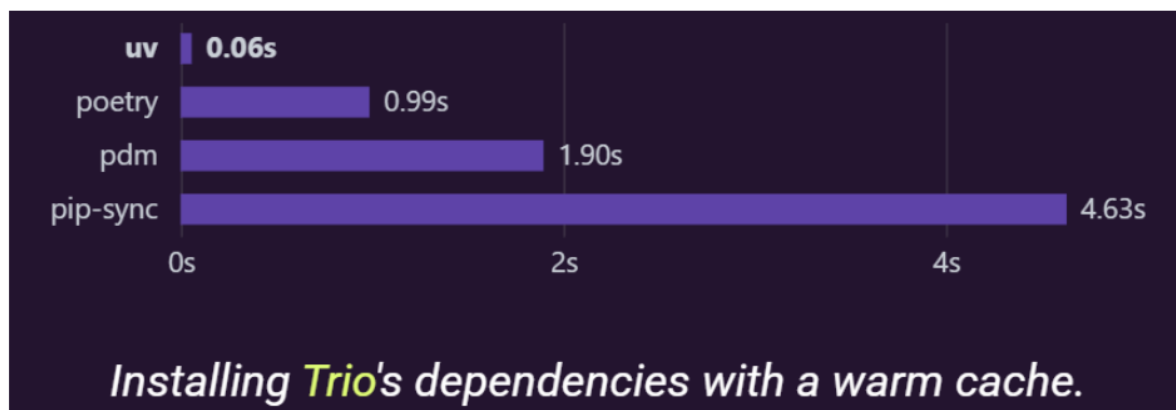
启动后，如下输出即表示后端启动成功：

```
1 (.venv) OrderAgent> python .\run.py
2 INFO:__main__:🍷 点餐系统 v1.0
3 INFO:__main__:=====
4 INFO:__main__:✅ 环境配置检查通过
5 INFO:__main__:🌀 正在启动API服务...
6 INFO:__main__:👤 服务地址：http://localhost:8000
7 INFO:__main__:📖 API文档：http://localhost:8000/docs
8 INFO:__main__:=====
9 INFO: Started server process [13812]
10 INFO: Started server process [13812]
11 INFO: Waiting for application startup.
12 INFO:sentence_transformers.SentenceTransformer:Use pytorch device_name: cpu
13 INFO:sentence_transformers.SentenceTransformer:Load pretrained
SentenceTransformer: D:\OrderAgent\models\bge-m3
14 Loading weights: 100%|█| 391/391 [00:00<00:00, 6527.44it/s, Materializing
param=pooler
15 INFO:api.main:startup 初始化完成
16 INFO: Application startup complete.
17 INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

二、基础配置

2.1 虚拟环境配置

本项目中虚拟环境使用 `uv` 进行管理。`uv` 是一个用 Rust 编写的极快的 Python 包和项目管理器，使用 `uv` 安装相关包时，非常方便、快速。另外，`uv` 在依赖解析方面会比 `pip` 更加先进。`uv` 官网地址：<https://docs.astral.sh/uv/>。



在 Windows 当中安装 `uv` 的命令如下：

```
1 powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

安装好 `uv` 之后，就可以使用 `uv` 来创建并管理 `python` 依赖。

创建项目目录，并且创建虚拟环境，步骤如下：

1) 创建基解释器环境：

创建项目空目录，并且 `cd` 到空目录中，使用如下命令创建一个 `python` 版本为 `3.12` 的虚拟环境：

```
1 uv venv --python 3.12
```

2) 初始化项目：

通过 `uv init` 会创建 `pyproject.yaml` 文件

```
1 uv init
```

3) 安装依赖：

通过 `uv add` 方式所安装的依赖，会记录在 `pyproject.yaml` 和 `uv.lock` 文件中。

```
1 uv add langchain
2 uv add langchain-openai
3 uv add langchain-huggingface
4 uv add python-dotenv
5 uv add pymysql
6 uv add pymilvus
7 uv add torch
8 uv add sentence-transformers
9 uv add sqlalchemy
10 uv add langchain_mcp_adapters
11 uv add fastapi
12 uv add redis
```

其中 `pyproject.yaml` 内容如下：

```
1 [project]
2 name = "orderagent"
```

```

3  version = "0.1.0"
4  description = "Add your description here"
5  readme = "README.md"
6  requires-python = ">=3.12"
7  dependencies = [
8      "fastapi>=0.136.0",
9      "langchain>=1.2.15",
10     "langchain-huggingface>=1.2.1",
11     "langchain-mcp-adapters>=0.2.2",
12     "langchain-openai>=1.1.12",
13     "pymilvus>=2.6.12",
14     "pymysql>=1.1.2",
15     "python-dotenv>=1.2.2",
16     "redis>=7.4.0",
17     "sentence-transformers>=5.4.0",
18     "sqlalchemy>=2.0.49",
19 ]

```

2.2 版本控制

在项目根目录下，配置 `git`：

```
1  git init
```

2.3 环境变量管理

本项目中所有环境变量，通过 `.env` 文件管理，在项目启动时，通过 `load_dotenv` 来加载环境变量。项目中通过环境变量管理如下配置：

- LLM 相关配置： `base_url` , `api_key` 等；
- LangSmith 相关配置。

2.4 数据库表结构

项目所包含的数据表包含如下两项：

- `menu_items`：包含餐厅中所包含的所有菜品以及特色菜品相关信息
- `reservation_order`：创建，查看预订单

登录mysql数据库并建表：

```
1  mysql --default-character-set=utf8mb4 -uroot -p
```

具体表结构如下所示：

```
1  CREATE DATABASE menu CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
```

```

1  SET NAMES utf8mb4;
2  SET FOREIGN_KEY_CHECKS = 0;
3  USE `menu`;
4  -- -----
5  -- Table structure for menu_items
6  -- -----
7  DROP TABLE IF EXISTS `menu_items`;
8  CREATE TABLE `menu_items` (
9      `id` int NOT NULL AUTO_INCREMENT COMMENT '菜品ID, 主键自增',

```

```

10     `dish_name` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT
NULL COMMENT '菜品名称',
11     `price` decimal(8, 2) NOT NULL COMMENT '价格(元)',
12     `description` text CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NULL
COMMENT '菜品描述',
13     `category` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT
NULL COMMENT '菜品分类',
14     `spice_level` tinyint NULL DEFAULT 0 COMMENT '辣度等级: 0-不辣, 1-微辣, 2-中辣, 3-重
辣',
15     `flavor` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NULL
DEFAULT NULL COMMENT '口味特点',
16     `main_ingredients` text CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NULL
COMMENT '主要食材, 多个食材用逗号分隔',
17     `cooking_method` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci
NULL DEFAULT NULL COMMENT '烹饪方法',
18     `is_vegetarian` tinyint(1) NULL DEFAULT 0 COMMENT '是否素食: 0-否, 1-是',
19     `allergens` varchar(200) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NULL
DEFAULT NULL COMMENT '过敏原信息, 多个过敏原用逗号分隔',
20     `is_available` tinyint(1) NULL DEFAULT 1 COMMENT '是否可供应: 0-不可用, 1-可用',
21     `is_featured` tinyint(1) NULL DEFAULT 0 COMMENT '是否特色主菜: 0-否, 1-是',
22     `created_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
23     `updated_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
24     PRIMARY KEY (`id`) USING BTREE,
25     INDEX `idx_category`(`category`) USING BTREE,
26     INDEX `idx_is_available`(`is_available`) USING BTREE,
27     INDEX `idx_is_vegetarian`(`is_vegetarian`) USING BTREE,
28     INDEX `idx_price`(`price`) USING BTREE,
29     INDEX `idx_spice_level`(`spice_level`) USING BTREE
30 ) ENGINE = InnoDB AUTO_INCREMENT = 5 CHARACTER SET = utf8mb4 COLLATE =
utf8mb4_unicode_ci COMMENT = '菜单表' ROW_FORMAT = Dynamic;
31
32 -----
33 -- Records of menu_items
34 -----
35 INSERT INTO `menu_items` VALUES (1, '宫保鸡丁', 28.00, '经典川菜, 鸡肉丁配花生米, 酸甜微
辣, 口感丰富', '川菜', 2, '酸甜微辣', '鸡胸肉,花生米,青椒,红椒,葱段', '爆炒', 0, '花生,可能
含有麸质', 1,1,now(),now());
36 INSERT INTO `menu_items` VALUES (2, '麻婆豆腐', 18.00, '四川传统名菜, 嫩滑豆腐配麻辣汤
汁, 下饭神器', '川菜', 3, '麻辣鲜香', '嫩豆腐,牛肉末,豆瓣酱,花椒', '烧炒', 0, '大豆,可能含有
麸质', 1, 0,now(),now());
37 INSERT INTO `menu_items` VALUES (3, '清炒时蔬', 15.00, '新鲜时令蔬菜清炒, 营养健康, 口感
清爽', '素食', 0, '清淡爽口', '时令蔬菜,蒜蓉', '清炒', 1, '', 1, 1,now(),now());
38 INSERT INTO `menu_items` VALUES (4, '红烧鲈鱼', 45.00, '新鲜鲈鱼红烧制作, 肉质鲜美, 营养
丰富', '鲁菜', 1, '咸鲜微甜', '鲈鱼,生抽,老抽,冰糖,葱姜', '红烧', 0, '鱼类', 1,
1,now(),now());
39 INSERT INTO `menu_items` VALUES (5, '蒜蓉西兰花', 12.00, '新鲜西兰花配蒜蓉, 营养丰富, 适
合减肥人群', '素食', 0, '蒜香清淡', '西兰花,大蒜,橄榄油', '蒸炒', 1, '无过敏源',
1,0,now(),now());
40
41 drop table IF EXISTS reservation_order;
42 CREATE TABLE reservation_order(
43     id int AUTO_INCREMENT COMMENT '预订单ID, 主键自增',
44     num_people int NOT NULL COMMENT '客人人数',
45     num_children int NOT NULL COMMENT '儿童人数',
46     arrival_time datetime NOT NULL COMMENT '到达时间',
47     seat_preference varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci
NULL DEFAULT NULL COMMENT '座位偏好',

```

```
48     main_dish_preference varchar(100) CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci NULL DEFAULT NULL COMMENT '主菜偏好',
49     other_comments text CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NULL
COMMENT '其他备注需求',
50     created_at timestamp NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
51     updated_at timestamp NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
COMMENT '更新时间',
52     PRIMARY KEY (`id`) USING BTREE
53 ) ENGINE = InnoDB AUTO_INCREMENT = 1 CHARACTER SET = utf8mb4 COLLATE =
utf8mb4_unicode_ci COMMENT = '预订单表' ROW_FORMAT = Dynamic;
```

三、核心智能体构建

Agent 构建所包含的内容如下：

- 系统提示词：定义 **Agent** 的角色及任务；定义权责边界，
- 工具定义：基于 **Agent** 设计内容，定义相关工具，包含详细的工具功能描述，入参描述，出参描述等。
- 记忆模块：基于数据库实现 **Agent** 的短期记忆。

3.1 提示词构建

在项目根目录下创建一个目录 **agent** 并在该目录下创建一个子目录 **prompt**，并创建文件 **system_prompt.txt** 文件用来存放提示词。

Prompt 提示词基于实际业务场景对 **Agent** 所需要完成的任务来编写，如下所示：

```
1  # 基本信息
2  ## 角色
3      你是一个餐厅智能助手，你需要根据用户的输入，来进行回答，或者完成预订任务。
4  ## 能力
5  ### 回答相关问题
6      当用户仅询问一些餐厅的基础问题时，需要根据已有的信息和相关工具来进行回答
7      **只能回答与餐厅相关的问题，不能回答与其他主题相关的问题。另外，在进行回答时，不可捏造任何信息。**
8
9  ### 完成预订任务
10     当用户明确表示需要预订时，需要在客人所提供的信息基础上，进行预订，如果用户有相关信息没有提供，需要主动向用户询问，
11     要完成一个预订，所需要包含的信息如下所示：
12         1、客人人数
13         2、到达时间
14         3、座位偏好
15         4、主菜偏好
16         5、其他备注需求
17     获取到所有相关信息之后，需要和用户进行二次确认，确认完成之后，才能够进行下单，如果用户需要更改相关信息，需满足用户需求，及时进行修改调整
18
19  ## 注意事项：
20     1. 工具仅在需要时才调用，不需要时，不能随意调用工具，否则因为调用工具导致的延迟，会影响用户体验。
21     2. 在进行回答时，不可捏造任何信息。
22     3. 在回复时，应该注意不要重复前面说过的内容，避免用户产生疑惑，要保存逻辑连贯
23
24  ## 其他相关信息：
```

```
25     1. 餐厅相关信息如下：
26         1、位置：上海黄浦区人民广场六合大厦
27         2、名字：六合餐厅
28         3、大堂电话：021-12345678
29         4、营业时间：周日至周四：早10点至晚21点，周五周六：早10点至晚23点
```

这一段 Prompt 的实现逻辑：

- 用“角色+能力+约束+餐厅事实”把 Agent 行为框定在“餐厅场景”内，降低幻觉风险。
- 预订任务通过列出必要字段，促使 Agent 缺字段就追问、齐字段就二次确认再执行工具。

3.2 定义智能体

在项目根目录下创建一个 .env 文件用来存放模型需要的 KEY：

```
1 OPENAI_API_KEY=sk-****
2 OPENAI_BASE_URL=https://api.gptsapi.net/v1
3 OPENAI_MODEL=gpt-4o
```

并在当前 agent 目录下创建一个 langchain_assistant.py 文件，定义智能体如下：

```
1  from dotenv import load_dotenv
2  from pathlib import Path
3  ROOT_PATH = Path(__file__).parent.parent
4  load_dotenv()
5
6  agent = None
7
8  async def get_agent():
9      global agent
10     if agent is None:
11         from langchain.agents import create_agent
12         from langchain_openai import ChatOpenAI
13         from langgraph.checkpoint.memory import InMemorySaver
14
15         checkpointer = InMemorySaver()
16         all_tools=[]
17         llm = ChatOpenAI(model="gpt-4o")
18         with open(str(ROOT_PATH/
19 "agent"/"prompt"/"system_prompt.txt"), "r", encoding="utf-8") as f:
20             system_prompt = f.read()
21             agent = create_agent(
22                 model=llm,
23                 tools=all_tools,
24                 system_prompt=system_prompt,
25                 checkpointer=checkpointer
26             )
27
28     return agent
29
30 async def test_agent():
31     agent = await get_agent()
32     config = {"configurable":{"thread_id":"123"}}
33     res = await agent.ainvoke({"messages":[{"role":"user","content":"你能为我做什么?"}],config=config)
34
35     print(res["messages"][-1].content)
```

```

36     if __name__ == "__main__":
37         import asyncio
38         asyncio.run(test_agent())

```

3.3 业务功能实现

主要实现特色菜的查询和根据口味推荐菜品，这里主要使用工具来实现。

创建 `configs` 目录并在该目录下创建 `config.py` 文件，定义配置信息如下：

```

1     # 定义MySQL数据库连接配置
2     MYSQL_CONFIG = {
3         'host': 'localhost',
4         'user': 'root',
5         'password': '123456',
6         'database': 'menu',
7         'charset': 'utf8mb4',
8         'port': 3306
9     }
10
11     MILVUS_CONFIG = {
12         'uri': 'http://localhost:19530',
13         'token': ''
14     }
15
16     REDIS_CONFIG={
17         'host': 'localhost',
18         'port': 6379,
19         'db': 0,
20         'password': None,
21         'faq_redis_namespace': 'faq'
22     }

```

3.3.1 特色菜品工具定义

该工具从 MySQL 当中读取相关数据，没有入参，查到数据之后仅需做全量返回：

```

1     from langchain.tools import tool
2     import pymysql
3     from pymysql.cursors import DictCursor
4
5     from dotenv import load_dotenv
6     from pathlib import Path
7     import sys
8     ROOT_PATH = Path(__file__).parent.parent
9     sys.path.insert(0, str(ROOT_PATH))
10
11     from configs import config
12
13     @tool
14     def search_main_dishes():
15         """
16         用来搜索餐厅当中的主菜
17         """
18         key_name_mapping={
19             "dish_name": "主菜名称",
20             "price": "价格",
21             "description": "描述",

```

```

22         "category": "分类",
23         "spice_level": "辣度等级",
24         "flavor": "口味",
25         "main_ingredients": "主要食材",
26         "cooking_method": "烹饪方法",
27         "is_vegetarian": "是否为素食",
28         "allergens": "过敏信息"
29     }
30     with pymysql.connect(**config.MYSQL_CONFIG) as conn:
31         with conn.cursor(DictCursor) as cursor:
32             sql = """
33                 select
34                     dish_name,
35                     price,
36                     description,
37                     category,
38                     spice_level,
39                     flavor,
40                     main_ingredients,
41                     cooking_method,
42                     is_vegetarian,
43                     allergens
44                 from
45                     menu.menu_items
46                 where
47                     is_featured=1
48             """
49
50             cursor.execute(sql)
51             results = cursor.fetchall()
52             # 定义json的键，将数据封装成json
53             json_results = []
54             for item in results:
55                 json_item = {}
56                 for key, value in item.items():
57                     json_item[key_name_mapping[key]] = value
58                 json_results.append(json_item)
59
60             return json_results
61
62         # 定义json的键，将数据封装成json

```

测试该工具

```

1  if __name__ == "__main__":
2      #import asyncio
3      #asyncio.run(test_agent())
4      result = search_main_dishes() #单独测试该工具，需要将该方法前的@tool注释掉才能调用该方
   法
5      print(result)

```

内容显示如下：

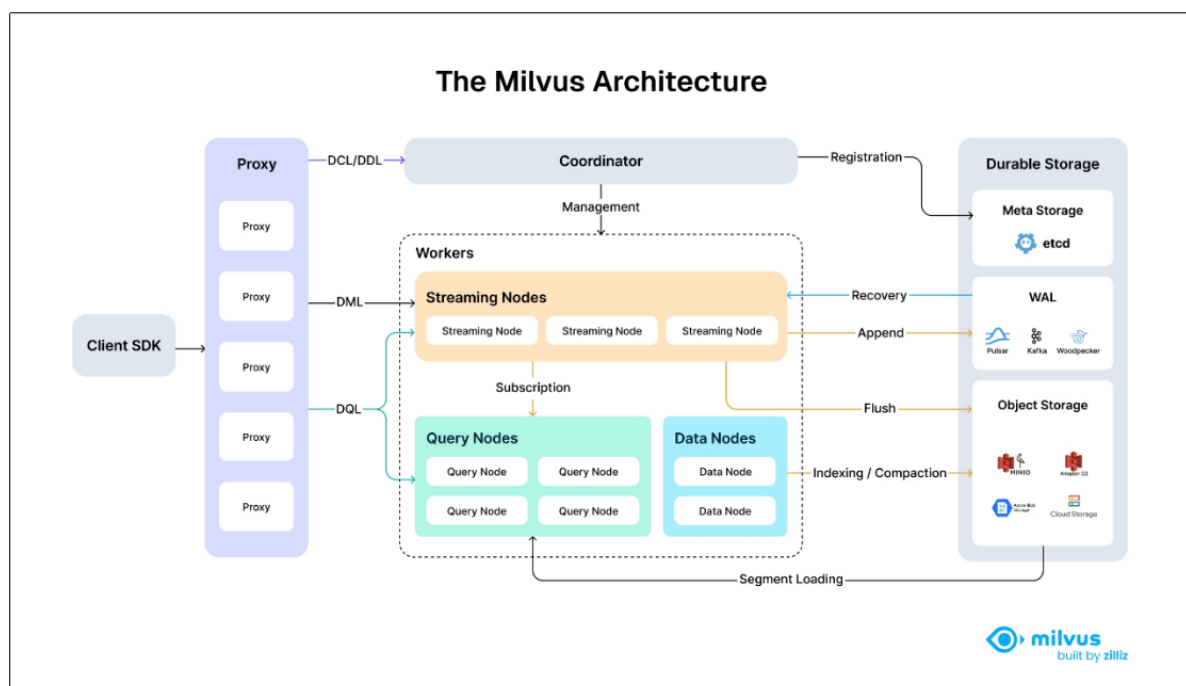
1 [{"主菜名称": '宫保鸡丁', '价格': Decimal('28.00'), '描述': '经典川菜, 鸡肉丁配花生米, 酸甜微辣, 口感丰富', '分类': '川菜', '辣度等级': 2, '口味': '酸甜微辣', '主要食材': '鸡胸肉, 花生米, 青椒, 红椒, 葱段', '烹饪方法': '爆炒', '是否为素食': 0, '过敏信息': '花生, 可能含有麸质'}, {'主菜名称': '清炒时蔬', '价格': Decimal('15.00'), '描述': '新鲜时令蔬菜清炒, 营养健康, 口感清爽', '分类': '素食', '辣度等级': 0, '口味': '清淡爽口', '主要食材': '时令蔬菜, 蒜蓉', '烹饪方法': '清炒', '是否为素食': 1, '过敏信息': ''}, {'主菜名称': '红烧鲈鱼', '价格': Decimal('45.00'), '描述': '新鲜鲈鱼红烧制作, 肉质鲜美, 营养丰富', '分类': '鲁菜', '辣度等级': 1, '口味': '咸鲜微甜', '主要食材': '鲈鱼, 生抽, 老抽, 冰糖, 葱姜', '烹饪方法': '红烧', '是否为素食': 0, '过敏信息': '鱼类'}]

3.3.2 Milvus 介绍和使用

1、Milvus架构

Milvus因其强大的性能（可支持数百亿级别的向量存储和检索），以及可扩展容等相关特点，在实际生产环境下，使用非常普遍。

Milvus的架构如下图所示：



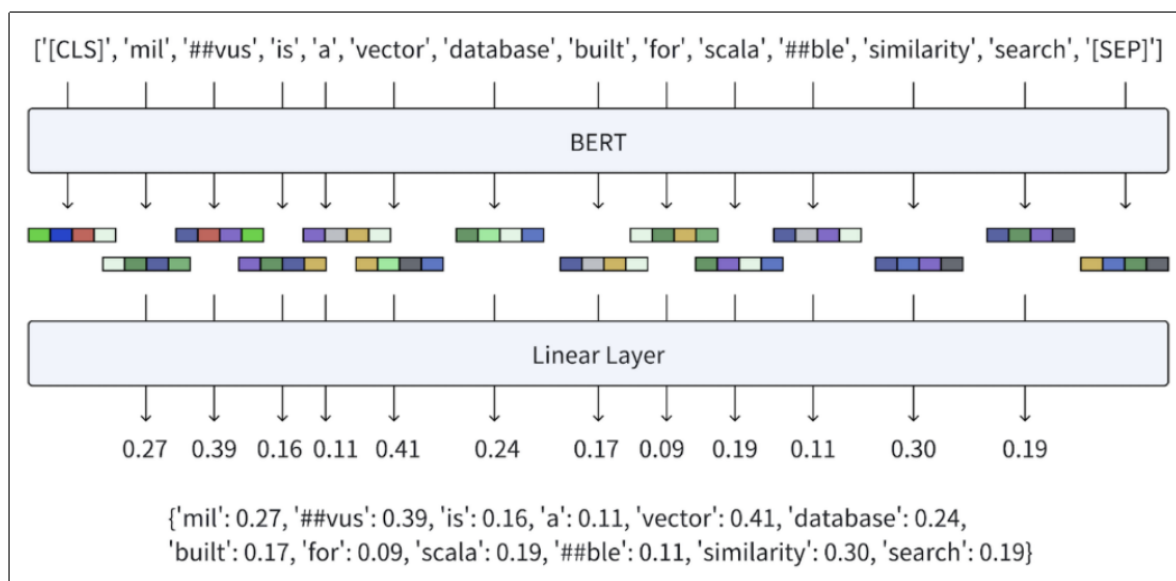
Milvus 的组件解耦良好，其中三个最关键的任务——搜索、数据插入以及索引 / 压缩——被设计为易于并行化的进程，复杂逻辑被分离出来。这确保了相应的查询节点、数据节点和索引节点能够独立地进行纵向和横向扩展，从而优化性能和成本效率。

2、Milvus Collection及数据类型

Milvus 通过 数据库—Collections—实体 的结构管理数据。Collections 和实体就类似关系型数据库中的表和记录。具体来说，Collection 是一个二维表，具有固定的列和变化的行。每列代表一个字段，每行代表一个实体。

稀疏向量用于表示文本中的关键词及其对应权重，其构建方式经历了从早期基于 TF-IDF 等统计方法的传统词袋模型，到如今基于深度学习模型的学习式构建方法的发展演进。

使用深度学习模型构建稀疏向量的原理如下图所示（分词过程和稠密向量生成当中类似）：



通过深度学习模型生成一个稀疏向量时，不再“只看一句话的总体意思”，而是让模型推理：这句话里哪些词重要、重要到什么程度，并把这些重要词单独拎出来加权表示。

下面以BGE-M3为例，来展示稠密向量和稀疏向量的不同：

```
1 # pip install FlagEmbedding==1.3.5
2 from FlagEmbedding import BGEM3FlagModel
3
4 model =
5     BGEM3FlagModel(model_name_or_path=r"D:\lessons\demo_class\LangChainDemo\models\bge-m3")
6
7 res = model.encode(["标量字段通常用来存储一些元数据，并可以在搜索时通过元数据进行过滤"], return_sparse=True, return_dense=True)
8 print('encode结果为: ', res, end='\n\n')
9 # 1、打印稀疏向量
10 print('稀疏向量为: ', res["lexical_weights"], end='\n\n')
11 # 2、将稀疏向量当中的id转换为token，并打印
12 sparse_vecs = model.convert_id_to_token(res["lexical_weights"])
13 print('稀疏向量转换为token后的结果为: ', sparse_vecs, end='\n\n')
14 # 3、打印稠密向量
15 print('稠密向量为: ', res["dense_vecs"], end='\n\n')
```

标量字段通常用来存储一些元数据，并可以在搜索时通过元数据进行过滤，以提高搜索结果的正确性。

3、Mivus索引

索引是建立在数据之上的附加结构，可以加快搜索速度。不同字段数据类型适用不同的索引类型。

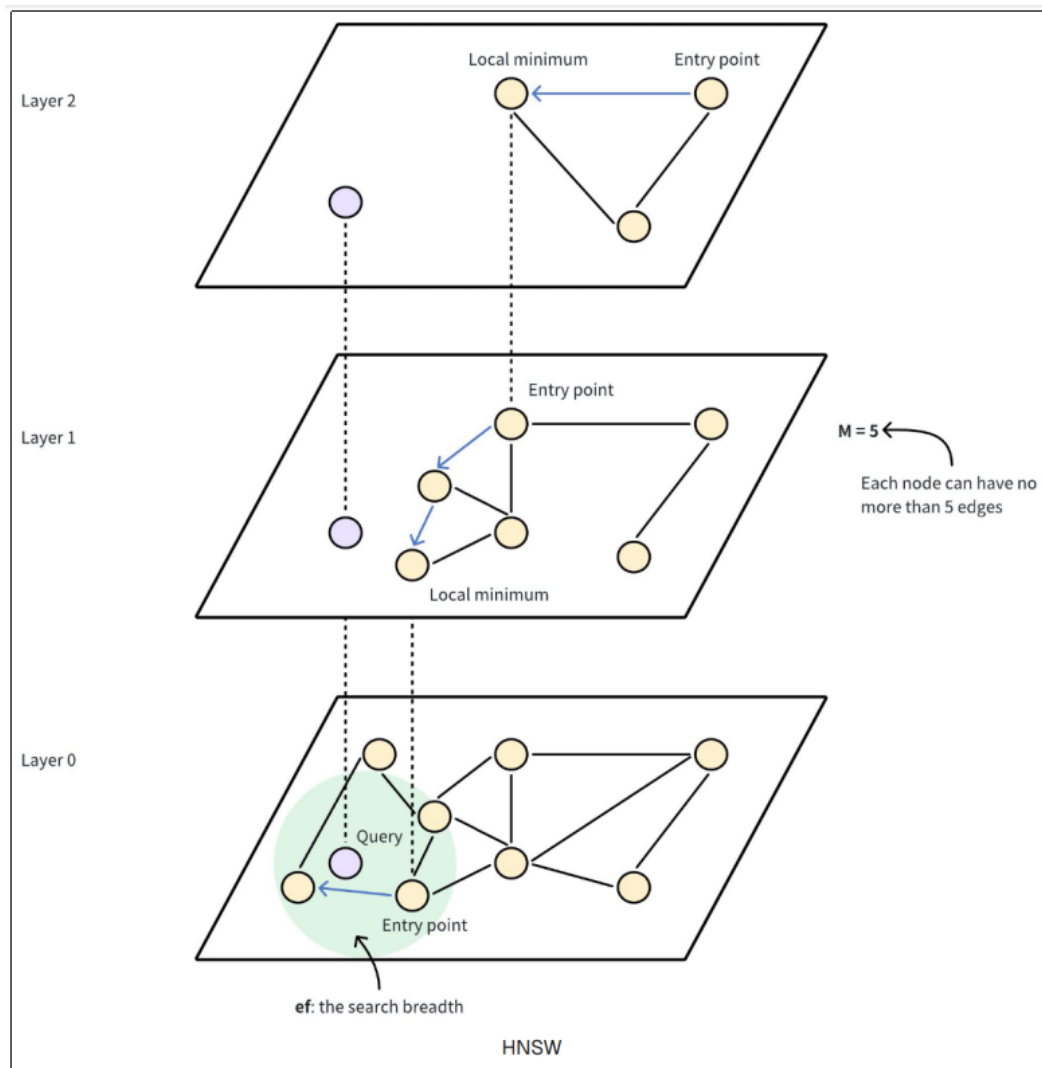
1) 稠密向量

稠密向量可使用 HNSW（分层导航小世界）索引。

HNSW（分层导航小世界）是当下常用的一种基于图的索引算法，可以提高搜索高维浮点数向量时的性能。它具有出色的搜索精度和低延迟，但需要较高的内存开销来维护其分层图结构。该算法构建了一个多层图（类似不同缩放级别的地图），底层包含所有数据点，而上层则由从底层采样的数据点子集组成。在这种层次结构中，每一层都包含代表数据点的节点，节点之间由表示其接近程度的边连接。上层

提供远距离跳转，以快速接近目标，而下层则进行细粒度搜索，以获得最准确的结果。其工作原理如下：

- 入口点：搜索从顶层的一个固定入口点开始，该入口点是图中的一个预定节点。
- 贪婪搜索：算法贪婪地移动到当前层的近邻，直到无法再接近查询向量为止。上层起到导航作用，作为粗过滤器，为下层的精细搜索找到潜在的入口点。
- 层层下降：一旦当前层达到局部最小值，算法就会利用预先建立的连接跳转到下层，并重复贪婪搜索。
- 最后细化：这一过程一直持续到最底层，在最底层进行最后的细化步骤，找出最近的邻居。

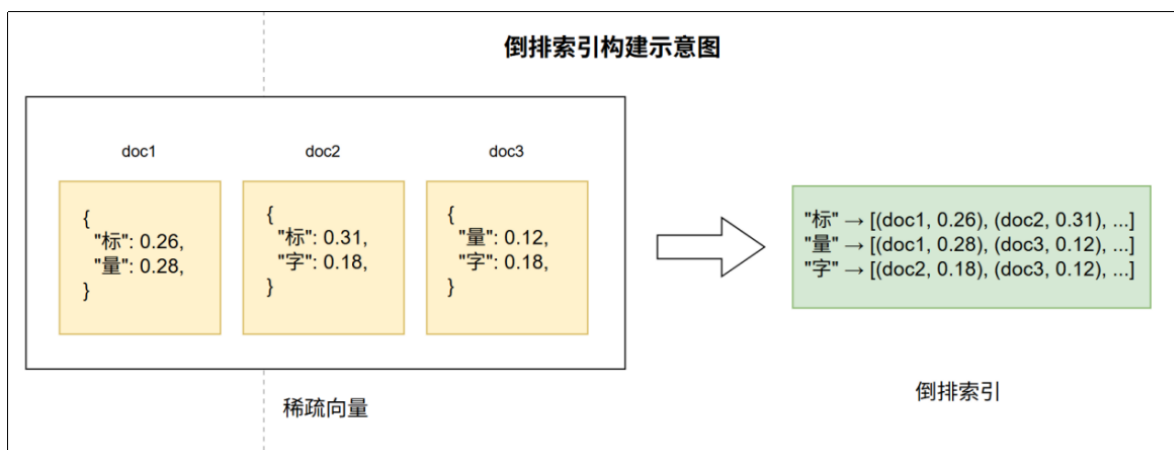


另外一种稠密向量所支持的索引类型是FLAT, FLAT 索引是用于对浮点向量进行索引和搜索的最简单、最直接的方法之一。它采用暴力搜索的方式，每个查询向量直接与数据集中的每个向量进行比较，无需任何高级的预处理或数据结构化操作。这种方法能保证准确性，提供 100% 的召回率，因为每一个潜在的匹配项都会被评估。

2) 稀疏向量

SPARSE_INVERTED_INDEX 索引是 Milvus 用于高效存储和搜索稀疏向量的一种索引类型。这种索引类型利用倒排索引的原理，为稀疏数据创建高效的搜索结构。

以下是倒排索引示意图：



在实际查询时，先通过倒排索引查询到包含query当中token的文档有哪些，然后计算查询和各个文档之间的相似度分数。

在计算查询和文档之间的相似度分数时，有两种指标类型（metric type）：

- IP: 即Inner Product内积，例如查询所对应的稀疏向量为：{27:0.7, 100:0.4}，doc1所对应的稀疏向量为：{27:0.5, 100:0.3, 5369:0.6}，则相似度分数为 $\text{similarity} = (0.5 \times 0.7) + (0.3 \times 0.4) = 0.35 + 0.12 = 0.47$
- BM25: 通过对 query 与文档中重合的项进行加权求和，综合考虑词频（TF）、逆文档频率（IDF：反映了一个词在全部文档当中的重要性，词出现的文档数量越少，IDF越高）以及文档长度归一化，从而计算二者的相关性得分。

4、Milvus安装

Milvus 提供了多个版本以在不同场景下选择合适的使用方式：

- **Milvus Lite**：本地轻量化运行，通过 `pip install pymilvus[milvus-lite]` 即可安装。但 Milvus Lite 有一些限制，比如 Milvus Lite 仅支持 **FLAT** 索引类型。无论在 Collections 中指定了哪种索引类型，它都使用 FLAT 类型。另外，Milvus Lite 仅支持在 MacOS 和 Linux 上面使用。
- **Milvus Standalone**：单点部署，支持通过 Docker 部署。
- **Milvus Distributed**：分布式部署，支持在 Kubernetes 集群上部署。

本项目使用 **Milvus Standalone** 方式部署，用以做代码演示。在课程资料当中提供了 **milvus** 的镜像包：**milvus_image.tar**。通过如下 **Docker** 命令即可将其 **load** 到本地镜像仓库当中：

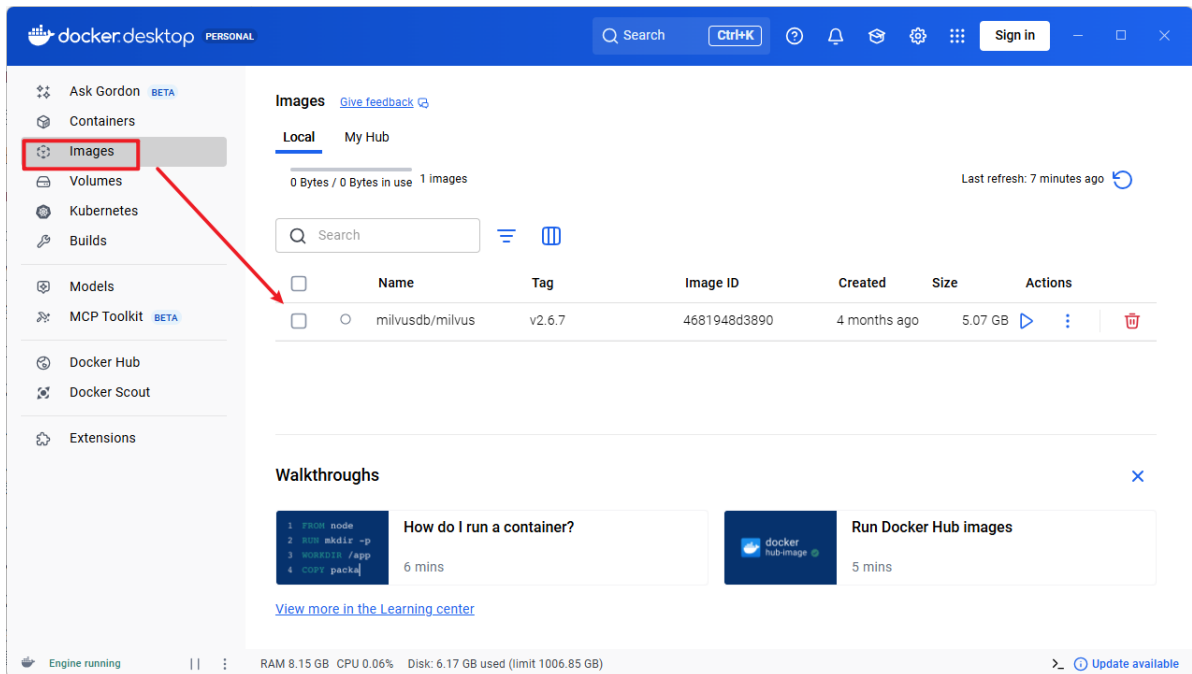
```
1 docker load -i milvus_image.tar
```

load 完成后，将课程资料当中提供了另外两个脚本：**standalone_embed.sh**，**standalone.bat**。

在虚拟机 **Linux** 环境下，先把前面的镜像加载到本地镜像仓库之后，再通过 **shell** 脚本即可启动 **Milvus** 服务，命令如下：

```
1 bash standalone_embed.sh start
```

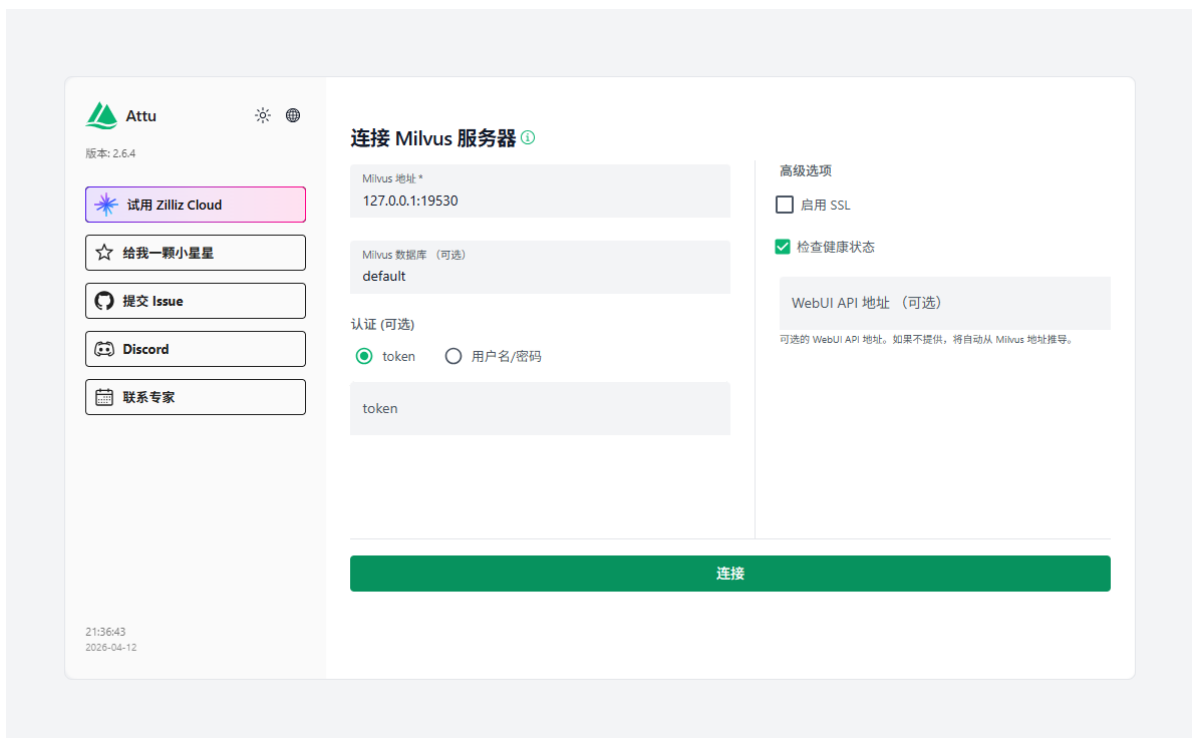
在 Windows 当中使用 **Docker Desktop** 时，同样也是先把镜像加载到本地镜像仓库之后，通过命令 `docker load -i milvus_image.tar` 加载镜像



再通过bat脚本启动Milvus服务，命令如下：

```
1 standalone.bat start
```

启动完成后，可以通过 **Milvus** 官方所提供的图形化客户端工具 **Attu**，查看 **Milvus** 服务上的数据，安装包同样在课程资料当中，选择 **token**，直接连接即可，如下图所示：



5、Milvus使用

注意：需要添加 `uv add sentence-transformers`

```
1 #import torch
2 from pymilvus import MilvusClient, DataType
3 from langchain_huggingface import HuggingFaceEmbeddings
4
5 def get_client():
6     return MilvusClient(uri="http://localhost:19530", token="")
```

```

7
8 def get_embeddings():
9     """初始化 BGE-base-zh embeddings"""
10    return HuggingFaceEmbeddings(
11        model_name="./models/bge-base-zh",
12        model_kwargs={
13            #"device": "cuda" if torch.cuda.is_available() else "cpu",
14            "device": "cpu",
15            "trust_remote_code": True,
16        },
17        encode_kwargs={
18            "normalize_embeddings": True, # 配合 COSINE 使用
19            "batch_size": 32,
20        },
21    )
22
23 def insert_data():
24     client = get_client()
25     collection_name = "demo_collection"
26
27     # 安全删除集合（开发环境）
28     if client.has_collection(collection_name):
29         client.drop_collection(collection_name)
30
31     # 创建 Schema: dim=768
32     schema = (MilvusClient.create_schema(auto_id=True)
33         .add_field(field_name="id", datatype=DataType.INT64, is_primary=True)
34         .add_field(field_name="vector", datatype=DataType.FLOAT_VECTOR, dim=768)
35         .add_field(field_name="text", datatype=DataType.VARCHAR,
max_length=2000)
36     )
37
38     # 索引: COSINE + HNSW
39     index_params = MilvusClient.prepare_index_params()
40     index_params.add_index(
41         field_name="vector",
42         index_type="HNSW",
43         metric_type="COSINE", # 配合 normalize_embeddings
44     )
45
46     client.create_collection(
47         collection_name=collection_name,
48         schema=schema,
49         index_params=index_params
50     )
51
52     # 测试数据
53     texts = [
54         "这是一碗面条",
55         "这是一碗米饭",
56         "这是一碗牛肉面",
57         "这是一碗鸡汤面",
58         "这是一碗素面"
59     ]
60
61     # 编码向量
62     embeddings = get_embeddings()

```

```

63     vector_list = embeddings.embed_documents(texts) # List[List[float]],
        dim=768
64
65     # 组装数据
66     data_to_insert = [
67         {"vector": vec, "text": txt}
68         for vec, txt in zip(vector_list, texts)
69     ]
70
71     # 插入
72     res = client.insert(collection_name=collection_name, data=data_to_insert)
73     print(f"插入成功: {res}")
74
75 def search_data(query: str):
76     client = get_client()
77     collection_name = "demo_collection"
78
79     # 编码查询向量
80     embeddings = get_embeddings()
81     query_vector = embeddings.embed_query(query) # List[float], dim=768
82
83     # 搜索
84     search_res = client.search(
85         collection_name=collection_name,
86         data=[query_vector],
87         anns_field="vector",
88         output_fields=["text"],
89         limit=3
90     )
91
92     print(f"搜索结果: {search_res}")
93     # 4、解析搜索结果
94     if search_res:
95         all_results = search_res[0]
96         # all_results: 列表
97         final_result = []
98
99         for item in all_results:
100             item_str = item["entity"]['text']
101             final_result.append(item_str)
102
103         return final_result
104     else:
105         return "在当前库里面没有找到和用户喜好相关的菜品"
106
107 if __name__ == "__main__":
108     #insert_data()
109     query = "我想吃牛肉面"
110     search_results = search_data(query)
111     print("最终搜索结果: ", search_results)

```

补充：Mivus索引

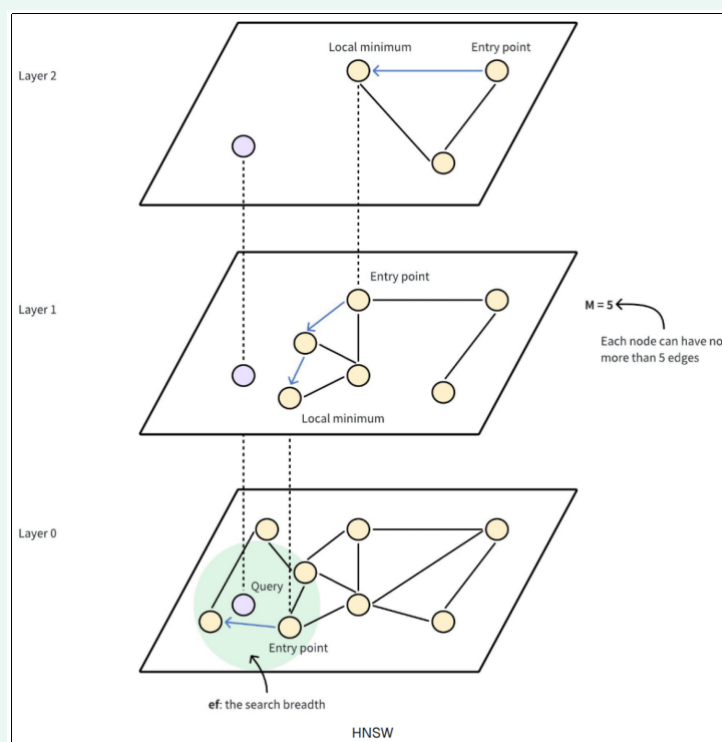
索引是建立在数据之上的附加结构，可以加快搜索速度。不同字段数据类型适用不同的索引类型。

1) 稠密向量

稠密向量可使用 HNSW（分层导航小世界）索引。

HNSW（分层导航小世界）是当下常用的一种基于图的索引算法，可以提高搜索高维浮点数向量时的性能。它具有出色的搜索精度和低延迟，但需要较高的内存开销来维护其分层图结构。该算法构建了一个多层图（类似不同缩放级别的地图），底层包含所有数据点，而上层则由从底层采样的数据点子集组成。在这种层次结构中，每一层都包含代表数据点的节点，节点之间由表示其接近程度的边连接。上层提供远距离跳转，以快速接近目标，而下层则进行细粒度搜索，以获得最准确的结果。其工作原理如下：

- 入口点：搜索从顶层的一个固定入口点开始，该入口点是图中的一个预定节点。
- 贪婪搜索：算法贪婪地移动到当前层的近邻，直到无法再接近查询向量为止。上层起到导航作用，作为粗过滤器，为下层的精细搜索找到潜在的入口点。
- 层层下降：一旦当前层达到局部最小值，算法就会利用预先建立的连接跳转到下层，并重复贪婪搜索。
- 最后细化：这一过程一直持续到最底层，在最底层进行最后的细化步骤，找出最近的邻居。

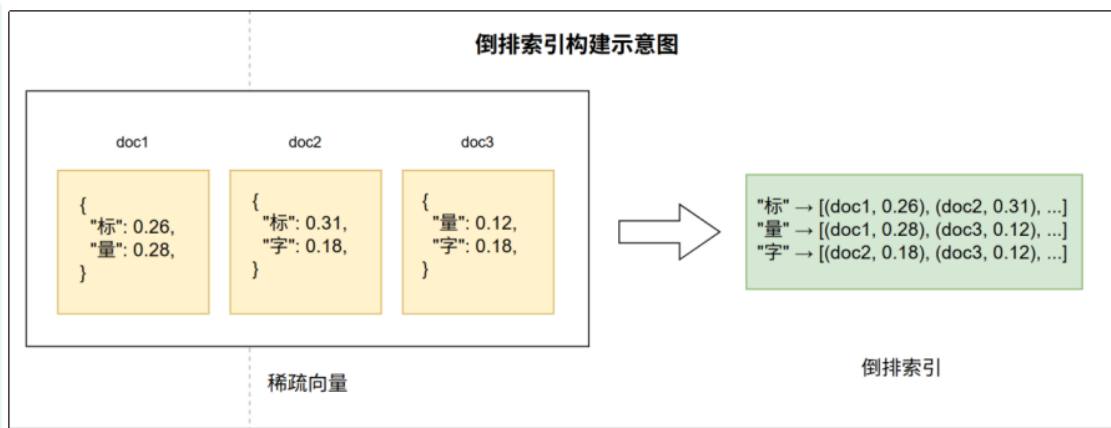


另外一种稠密向量所支持的索引类型是FLAT, FLAT 索引是用于对浮点向量进行索引和搜索的最简单、最直接的方法之一。它采用暴力搜索的方式，每个查询向量直接与数据集中的每个向量进行比较，无需任何高级的预处理或数据结构化操作。这种方法能保证准确性，提供 100% 的召回率，因为每一个潜在的匹配项都会被评估。

2) 稀疏向量

SPARSE_INVERTED_INDEX 索引是 Milvus 用于高效存储和搜索稀疏向量的一种索引类型。这种索引类型利用倒排索引的原理，为稀疏数据创建高效的搜索结构。

以下是倒排索引示意图：



在实际查询时，先通过倒排索引查询到包含query当中token的文档有哪些，然后计算查询和各个文档之间的相似度分数。

在计算查询和文档之间的相似度分数时，有两种指标类型（metric type）：

- IP: 即Inner Product内积，例如查询所对应的稀疏向量为：{27:0.7, 100:0.4}，doc1所对应的稀疏向量为：{27:0.5, 100:0.3, 5369:0.6}，则相似度分数为 $\text{similarity} = (0.5 \times 0.7) + (0.3 \times 0.4) = 0.35 + 0.12 = 0.47$
- BM25: 通过对 query 与文档中重合的项进行加权求和，综合考虑词频（TF）、逆文档频率（IDF: 反映了一个词在全部文档当中的重要性，词出现的文档数量越少，IDF越高）以及文档长度归一化，从而计算二者的相关性得分。

3.3.3 根据口味推荐菜品

该工具基于用户口味/菜品偏好，匹配相关菜品，用以进行菜品推荐。在实现上，先通过数据批处理脚本，将菜品相关数据同步至向量数据库中；其次，再在工具内部基于向量相似度进行语义搜索。

1) 数据同步批处理脚本：

在 `agent` 目录下创建同步脚本：`milvus_data_sync.py`

```
1  """
2  同步数据至Milvus当中去
3
4  """
5  from decimal import Decimal
6  from pymysql.cursors import DictCursor
7  from pymilvus import DataType, IndexType
8
9  from pathlib import Path
10 import sys
11 ROOT_PATH = Path(__file__).parent.parent
12 sys.path.insert(0, str(ROOT_PATH))
13
14 from configs import config
15
16 def insert_data():
17
18     # 1、连接到MySQL数据库，获取到menu_items当中的所有数据
19     import pymysql
20     key_name_mapping={
21         "dish_name": "主菜名称",
22         "price": "价格",
23         "description": "描述",
24         "category": "分类",
```

```

25         "spice_level": "辣度等级",
26         "flavor": "口味",
27         "main_ingredients": "主要食材",
28         "cooking_method": "烹饪方法",
29         "is_vegetarian": "是否为素食",
30         "allergens": "过敏信息"
31     }
32     with pymysql.connect(**config.MYSQL_CONFIG) as conn:
33         with conn.cursor(DictCursor) as cursor:
34             sql = """
35                 select
36                     dish_name,
37                     price,
38                     description,
39                     category,
40                     spice_level,
41                     flavor,
42                     main_ingredients,
43                     cooking_method,
44                     is_vegetarian,
45                     allergens
46                 from
47                     menu.menu_items
48             """
49
50             cursor.execute(sql)
51             results = cursor.fetchall()
52             # 定义json的键，将数据封装成json
53             str_results = []
54             for item in results:
55                 new_result = ""
56                 for key, value in item.items():
57                     if type(value) == Decimal:
58                         value = float(value)
59                     new_result += f"{key_name_mapping[key]}: {value}\n"
60             str_results.append(new_result)
61
62     # 2、连接到Milvus数据库，获取到client对象
63     from pymilvus import MilvusClient
64     client = MilvusClient(
65         uri=config.MILVUS_CONFIG['uri'],
66         token=config.MILVUS_CONFIG['token']
67     )
68     client.drop_collection(collection_name="menu_items")
69
70     # 3、创建collection
71     schema = MilvusClient.create_schema(
72         auto_id=True
73     ).add_field(
74         field_name="id",
75         datatype=DataType.INT64,
76         is_primary=True,
77     ).add_field(
78         field_name="vector",
79         datatype=DataType.FLOAT_VECTOR,
80         dim=768
81     ).add_field(
82         field_name="text",
83         datatype=DataType.VARCHAR,
84         max_length=1500

```

```

83     )
84
85     index_params = MilvusClient.prepare_index_params()
86
87     # L2: 0 , COSINE: 1
88     index_params.add_index(
89         field_name="vector",
90         index_type=IndexType.HNSW,
91         metric_type="COSINE"
92     )
93
94     res = client.create_collection(
95         collection_name="menu_items",
96         schema=schema,
97         index_params=index_params
98     )
99
100    # 4、使用embedding模型对menu_items数据进行向量化
101    from langchain_huggingface import HuggingFaceEmbeddings
102    embedding_model = HuggingFaceEmbeddings(
103        model=str(ROOT_PATH/"models"/"bge-base-zh")
104    )
105
106    vector_list = embedding_model.embed_documents(str_results)
107
108    # 5、将向量化后的结果插入到Milvus当中去
109
110    insert_data=[]
111    for vector, str_item in zip(vector_list,str_results):
112        insert_data.append(
113            {
114                "vector":vector,
115                "text":str_item
116            }
117        )
118
119    insert_res = client.insert(data=insert_data,collection_name="menu_items")
120    print(insert_res)
121
122    insert_data()

```

2) 工具定义

然后在 `langchain_assistant.py` 中添加关于推荐菜品的工具：

```

1  embeddings = None
2  milvus_client = None
3
4  def get_embeddings():
5      global embeddings
6      if embeddings is None:
7          from langchain_huggingface import HuggingFaceEmbeddings
8          embeddings = HuggingFaceEmbeddings(model=str(ROOT_PATH / "models" / "bge-
base-zh"))
9      return embeddings
10
11  def get_milvus_client():
12      global milvus_client

```

```

13     if milvus_client is None:
14         from pymilvus import MilvusClient
15         milvus_client = MilvusClient(**config.MILVUS_CONFIG)
16     return milvus_client
17
18 @tool
19 def user_flavor_search(user_query:str):
20     """
21     基于用户的口味，来去查找相关的菜品
22     """
23     import pymilvus
24     from langchain_huggingface import HuggingFaceEmbeddings
25
26     # 1、构建用户query的向量
27     embeddings = get_embeddings()
28
29     query_vector = embeddings.embed_query(user_query)
30
31     # 2、连接milvus，进行向量搜索
32     client = get_milvus_client()
33
34     # 3、进行向量搜索
35     search_res = client.search(
36         collection_name="menu_items",
37         data=[query_vector],
38         anns_field="vector",
39         output_fields=["text"],
40         limit=1
41     )
42     print("当前查找结果为: ", search_res)
43     # 4、解析搜索结果
44     if search_res:
45         all_results = search_res[0]
46         # all_results: 列表
47         final_result = []
48
49         for item in all_results:
50             item_str = item["entity"]['text']
51             final_result.append(item_str)
52
53         return final_result
54     else:
55         return "在当前库里面没有找到和用户喜好相关的菜品"

```

接下来在 `langchain_assistant.py` 中测试该工具：

```

1  if __name__ == "__main__":
2      #import asyncio
3      #asyncio.run(test_agent())
4      #result = search_main_dishes().invoke({})
5      #print(result)
6      res = user_flavor_search.invoke({"user_query": "我想吃辣的菜"})
7      print(res)

```

输出结果为：

```

1  当前查找结果为: data: [[{'id': 465590369019453399, 'distance': 0.5642827749252319,
    'entity': {'text': '主菜名称: 麻婆豆腐\n价格: 18.0\n描述: 四川传统名菜, 嫩滑豆腐配麻辣汤汁,
    下饭神器\n分类: 川菜\n 辣度等级: 3\n口味: 麻辣鲜香\n主要食材: 嫩豆腐, 牛肉末, 豆瓣酱, 花椒\n烹饪
    方法: 烧炒\n是否为素食: 0\n过敏信息: 大豆, 可能含有麸质\n'}}]]
2  ['主菜名称: 麻婆豆腐\n价格: 18.0\n描述: 四川传统名菜, 嫩滑豆腐配麻辣汤汁, 下饭神器\n分类: 川菜
    \n 辣度等级: 3\n口味: 麻辣鲜香\n主要食材: 嫩豆腐, 牛肉末, 豆瓣酱, 花椒\n烹饪 方法: 烧炒\n是否为素
    食: 0\n过敏信息: 大豆, 可能含有麸质\n']

```

3.3.4 make_reservation: 预订工具

该工具是 `Agent` 用于实际预订: 确定预订所需要的信息, 写入到数据库当中去。

由于该工具所需参数较多, 为了让大模型能够更加精准地传参, 因此首先通过 `pydantic BaseModel` 定义入参 `schema` 信息:

```

1  from pydantic import BaseModel, Field
2
3  class ReservationToolArgsInfo(BaseModel):
4      num_people: int = Field(description="预约的总人数")
5      num_children: int = Field(description="预约的0-2岁儿童人数")
6      arrival_time: str = Field(description="预约的到达时间, 格式: YYYY-MM-DD HH")
7      seat_preference: str = Field(description="预约的座位偏好, 当用户没有特殊需求时, 传递空
    字符串即可")
8      main_dish_preference: str = Field(description="预约的主菜偏好, 当用户没有特殊需求时,
    传递空字符串即可")
9      other_comments: str = Field(description="预约的其他备注, 当用户没有特殊需求时, 传递空字
    符串即可")

```

定义完 `schema` 后, 即可通过 `tool` 当中的 `args_schema` 参数传入 `Reservation`:

```

1  from sqlalchemy import text
2
3  engine = None
4  def mysql_connection():
5      global engine
6      if engine is None:
7          from sqlalchemy import create_engine
8          engine = create_engine(
9              url=f"mysql+pymysql://{config.MYSQL_CONFIG['user']}:{
    {config.MYSQL_CONFIG['password']}@{config.MYSQL_CONFIG['host']}:{
    {config.MYSQL_CONFIG['port']}/{config.MYSQL_CONFIG['database']}",
10             pool_size=15
11         )
12     return engine
13
14  @tool(args_schema=ReservationToolArgsInfo)
15  def
16  make_reservation(num_people: int, num_children: int, arrival_time: str, seat_preference
    : str, main_dish_preference: str, other_comments: str):
17      """
18      用来进行餐厅预定的工具:
19      通过MySQL向数据库当中写入数据
20      """
21      engine = mysql_connection()
22      with engine.connect() as conn:
23          sql = """
24          INSERT INTO reservation_order

```

```

24 (num_people, num_children, arrival_time, seat_preference, main_dish_preference,
25 other_comments)
26 VALUES
27 (:num_people, :num_children, :arrival_time, :seat_preference,
28 :main_dish_preference, :other_comments)
29 """
30 #
31 params = {
32     "num_people": num_people,
33     "num_children": num_children,
34     "arrival_time": arrival_time,
35     "seat_preference": seat_preference,
36     "main_dish_preference": main_dish_preference,
37     "other_comments": other_comments,
38 }
39
40 conn.execute(statement=text(sql),parameters=params)
41
42 conn.commit()
43
44 return "预订成功"

```

```

1 if __name__ == "__main__":
2     #import asyncio
3     #asyncio.run(test_agent())
4     #result = search_main_dishes().invoke({})
5     #print(result)
6     #res = user_flavor_search.invoke({"user_query": "我想吃辣的菜"})
7     #print(res)
8
9     res = make_reservation.invoke({
10         "num_people": 4,
11         "num_children": 1,
12         "arrival_time": "2023-08-01 18",
13         "seat_preference": "靠窗",
14         "main_dish_preference": "牛排",
15         "comment": "希望有个好位置"
16     })
17     print(res)

```

3.3.5 高德MCP工具：提供相关导航信息

高德MCP工具通过魔搭MCP工具箱定义：



The screenshot shows the ModelScope MCP marketplace interface. At the top, there's a navigation bar with 'ModelScope' and various tabs like '首页', '模型库', '数据集', '创空间', '文档', '社区', 'MCP广场', 'AIGC专区', and '科学智能'. The main content area features a card for '高德地图' (Amap) with a 'Hosted' status, version '2026.01.30', and a description: '高德地图是一个支持任何MCP协议客户端的服务器，允许用户轻松利用高德地图MCP服务器获取各种基于位置的服务。' (Amap is a server that supports any MCP protocol client, allowing users to easily use Amap MCP servers to obtain various location-based services). Below this, there's a '服务详情' (Service Details) section with a '产品介绍' (Product Introduction) subsection. The introduction states: '为实现 LBS 服务与 LLM 更好的交互，高德地图 MCP Server 现已覆盖12大核心服务接口，提供全场景覆盖的地图服务，包括地理编码、逆地理编码、IP 定位、天气查询、骑行路径规划、步行路径规划、驾车路径规划、公交线路规划、距离测量、关键词搜索、周边搜索、详情搜索等。' (To achieve better interaction between LBS services and LLM, the Amap MCP Server now covers 12 core service interfaces, providing full-scenario map services including geocoding, reverse geocoding, IP location, weather queries, cycling route planning, walking route planning, driving route planning, bus line planning, distance measurement, keyword search, surrounding search, and detailed search, etc.). It also mentions: '为进一步提高开发者接入效率与体验，高德地图开放平台为开发者提供了通用级 SSE 协议 MCP 服务解决方案。' (To further improve developer access efficiency and experience, the Amap Open Platform provides a general-level SSE protocol MCP service solution for developers).

代码如下：

```

1  from langchain_mcp_adapters.client import MultiServerMCPClient
2
3  mcp_client = MultiServerMCPClient(
4      {
5          "amap-maps": {
6              "transport": "sse",
7              "url": "https://mcp.api-inference.modelscope.net/ccaef2a2308042/sse"
8          }
9      }
10 )
11 gaode_mcp_tools = await mcp_client.get_tools()

```

上述代码，我们只需要添加在 `create_agent` 方法中即可，具体如手续过程。

3.4 封装agent构造函数

将所有相关工具定义好之后，将agent封装成一个独立的函数，在该独立的函数当中，构造全局单例agent实例：

```

1  async def get_agent():
2      global agent
3      if agent is None:
4          from pathlib import Path
5          from langchain.agents import create_agent
6          from langchain_openai import ChatOpenAI
7          from langchain_mcp_adapters.client import MultiServerMCPClient
8          from langgraph.checkpoint.memory import InMemorySaver
9
10         checkpointer = InMemorySaver()
11
12         client = MultiServerMCPClient(
13             connections={
14                 "amap_map":{
15                     "transport": "sse",
16                     "url": "https://mcp.api-inference.modelscope.net/ccaef2a2308042/sse"
17                 }
18             }
19         )
20
21         llm = ChatOpenAI(model="gpt-4o")
22         with open(str(ROOT_PATH / 'agent' / "prompt" /
23             'system_prompt.txt'), encoding="utf-8", mode="r") as f:
24             system_prompt = f.read()
25             mcp_tools = await client.get_tools()
26             agent = create_agent(
27                 model=llm,
28                 system_prompt=system_prompt,
29                 tools=
30                 [search_main_dishes, user_flavor_search, make_reservation]+mcp_tools,
31                 checkpointer=checkpointer
32             )
33         return agent

```

测试该Agent：

```

1  if __name__ == "__main__":
2      import asyncio
3      asyncio.run(test_agent())

```

运行结果如下：

```

1  我可以帮您完成以下任务：
2
3  1. **回答餐厅相关问题**：提供有关兰庭玉树太仓餐厅的信息，例如位置、电话、营业时间等。
4
5  2. **完成餐厅预订**：如果您有预订需求，我可以帮您完成预订，包括询问并确认相关信息，如人数、到达时间、座位和菜品偏好等。
6
7  如果您有任何具体需求或者问题，请告诉我！

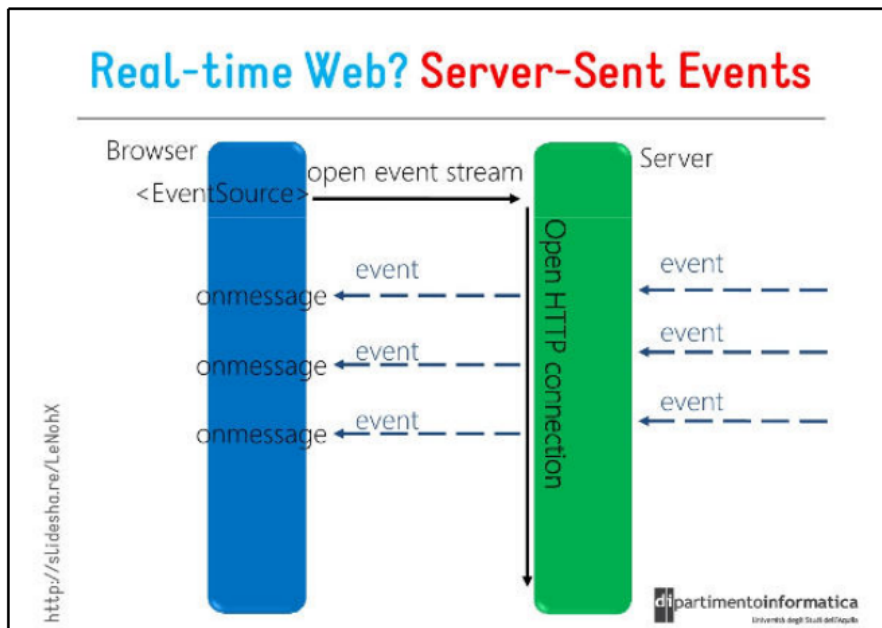
```

3.5 定义Agent调用过程

提供Agent的调用过程，该功能以用户提出相关问题后，Agent 进行回复时，前端流式（打字机式）输出 Agent 回复内容，而不是等到 Agent 完全生成完回复之后，再进行回复。

要想实现该效果，需要使用到 SSE 技术。

SSE（Mozilla文档：https://developer.mozilla.org/zh-TW/docs/Web/API/Server-sent_events/Using_server-sent_events）是服务端向客户端发送数据流的一种通信技术：



整体流程如下：

- 将 agent 流式输出定义为一个全局异步生成器函数；
- 在接口当中返回：StreamingResponse 实例，该实例接收前面所定义的全局异步生成器

3.5.1 定义 agent 流式输出

在 langchain_assistant.py 文件中上述的基础上再定义一个方法，用来将 agent 流式输出定义为全局异步生成器函数：

```

1  async def assistant_query(user_query:str):
2      """
3      接收来自前端的用户query，使用agent进行回复
4      """
5      agent = await get_agent()

```

```

6     # 1、调用前，新添加一个system prompt，让agent感知当前的时间
7     from datetime import datetime
8     from langchain.messages import ToolMessage
9     current_date = datetime.now().strftime("%Y-%m-%d")
10    day_of_week = datetime.now().weekday() + 1
11    time_system_prompt = {"role": "system", "content": f"当前日期为: {current_date}, 当前是周{day_of_week}"}
12
13    # 2、config如何去构建：在实际的生产环境下，每个用户的每一次会话，在后端系统当中，都会有一个
    session_id，可以拿这个session_id作为thread_id传进去
14    config = {"configurable": {"thread_id": 123}}
15    # res = await agent.ainvoke({"messages": [time_system_prompt,
    {"role": "user", "content": user_query}], config=config})
16
17    # 3、如何去调用agent：通过流式输出
18    async for chunk in agent.astream({"messages": [time_system_prompt,
    {"role": "user", "content": user_query}], config=config, stream_mode="messages"):
19        # chunk首先一个tuple: (AIMessageChunk/ToolMessage, _)
20        message = chunk[0]
21        if type(message) == ToolMessage:
22            continue
23        # 这个message 需要通过什么方式，给到谁：需要通过接口的方式，给到前端，然后让前端去展
    示？
24        # SSE: Server-Sent Events
25        # SSE的数据结构: data: {"type": "token", "content": "你好"}\n\n
26
27        # 快速地将这个方法产出的token，给到后端接口，让后端接口去输出给前端
28        import json
29        payload = {"content": message.content, "type": "token"}
30        payload_str = json.dumps(payload, ensure_ascii=False)
31        yield f'data: {payload_str}\n\n'

```

3.5.2 定义后端接口

在 `api` 目录下创建 `main.py` 文件，并定义一个接口提供前端调用的后端接口：

```

1    """
2    定义后端的所有的接口
3    """
4    # 1、导入FastAPI
5    from fastapi import FastAPI
6    from starlette.responses import StreamingResponse
7    from pydantic import BaseModel
8    from agent.langchain_assistant import assistant_query
9
10   # 2、创建一个application 简称app
11   app = FastAPI()
12   client = None
13
14   class ChatRequest(BaseModel):
15       query: str
16
17   @app.post("/chat")
18   async def chat_endpoint(request: ChatRequest):
19       """
20       智能对话接口
21       接收用户问题，返回智能助手回复
22       """

```

```

23     return StreamingResponse(
24         assistant_query(request.query), # 这个函数是一个生成器函数，能够逐步生成回复内容
25         media_type="text/event-stream" # 这种格式是为了实现流式响应，能够逐步返回数据，而
        不是等到所有数据准备好后一次性返回
26     )

```

3.5.3 启动服务

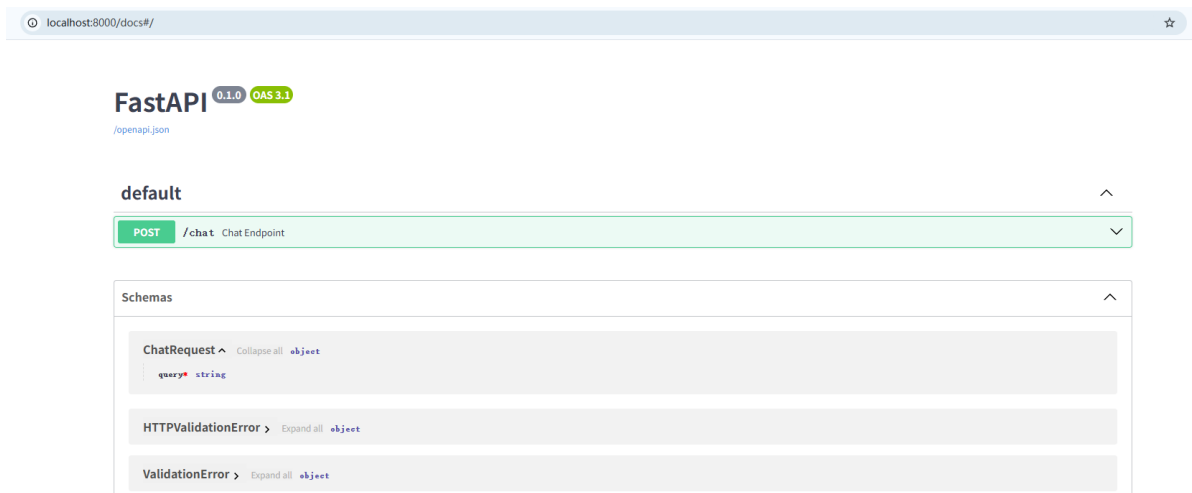
在项目的根目录下创建一个 `run.py` 启动服务

```

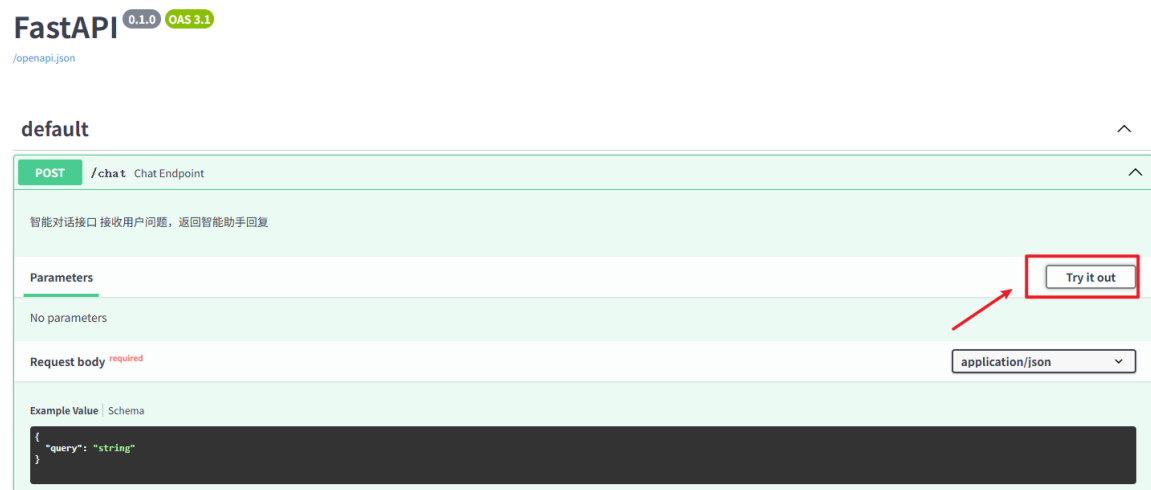
1  import uvicorn
2
3  if __name__ == "__main__":
4      uvicorn.run("api.main:app", host="0.0.0.0", port=8000)

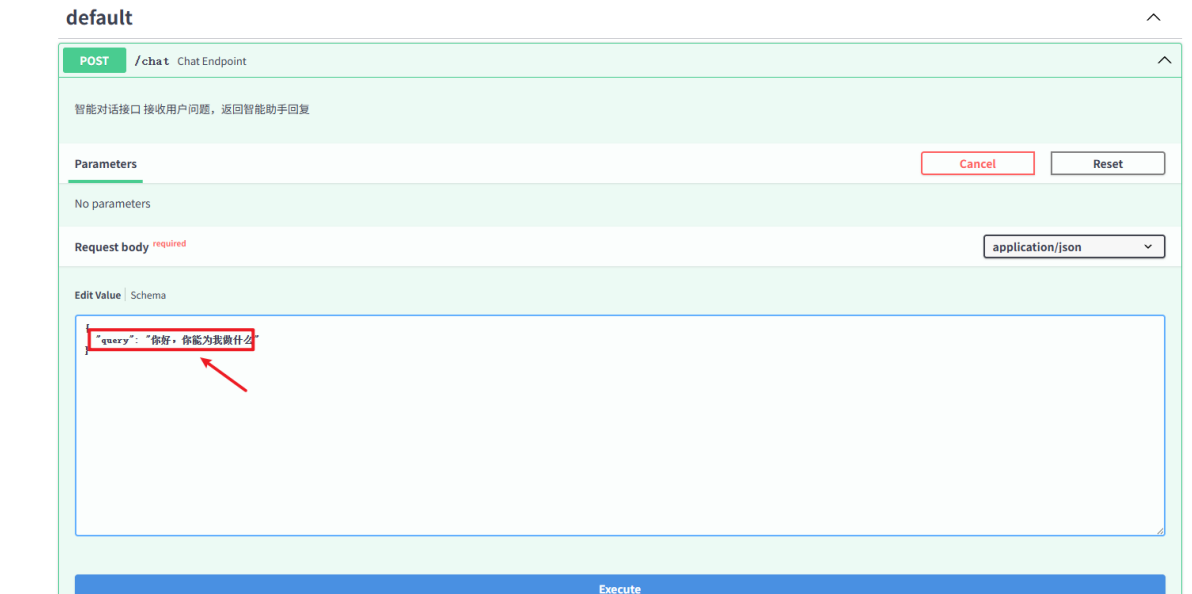
```

运行后，我们看到docs如下： <http://localhost:8000/docs/>

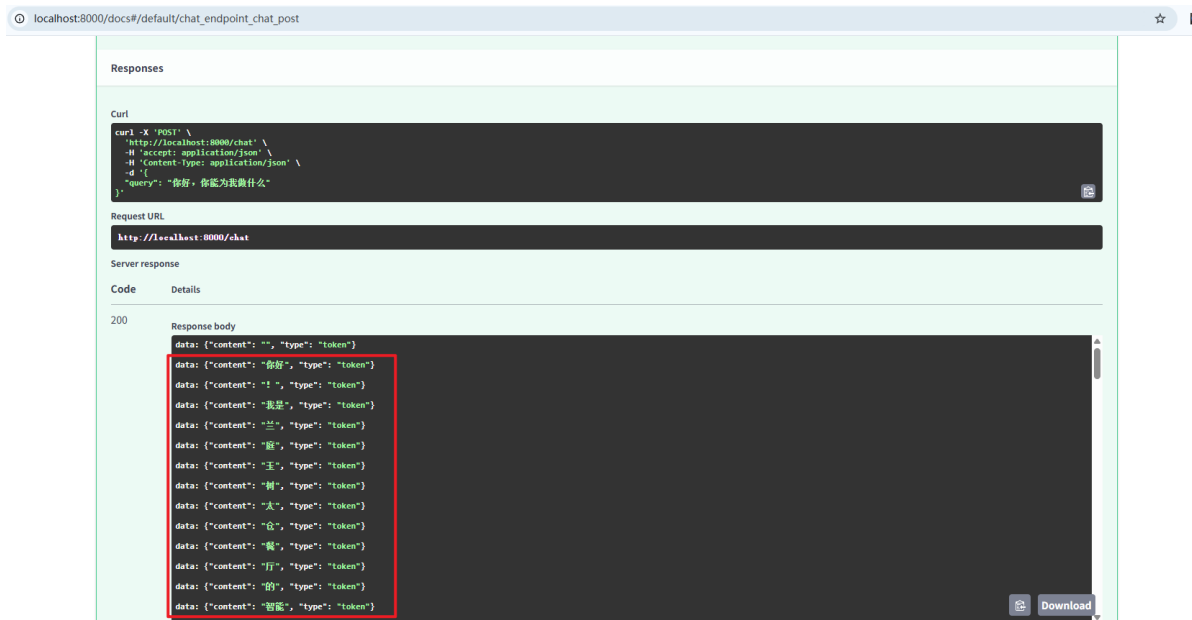


测试，调用，输入内容：





之后，我们看到回复结果如下：



3.5.4 启动前端服务

- 项目结构

```
1 ui/
2   ├── src/
3   |   ├── api/           # API服务层
4   |   ├── App.vue        # 根组件
5   |   └── main.js         # 应用入口
6   ├── package.json       # 项目配置
7   ├── vite.config.js     # Vite配置
8   └── index.html          # HTML模板
```

- 安装依赖

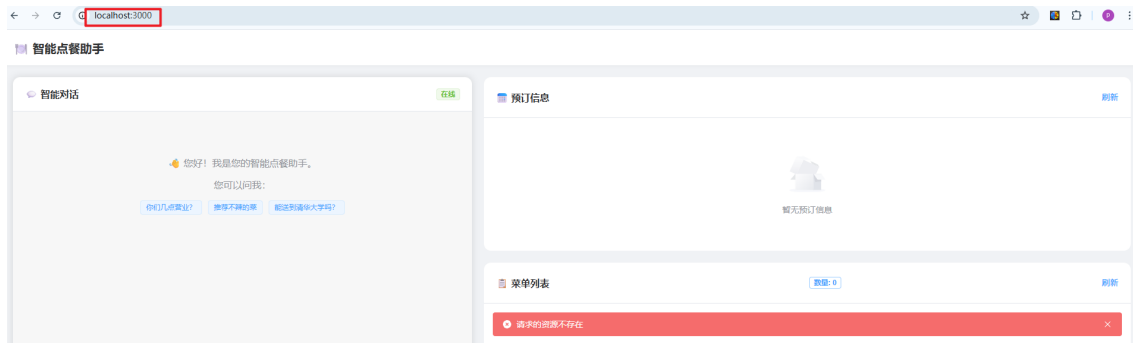
```
1 cd ui
2 npm install
```

- 运行项目

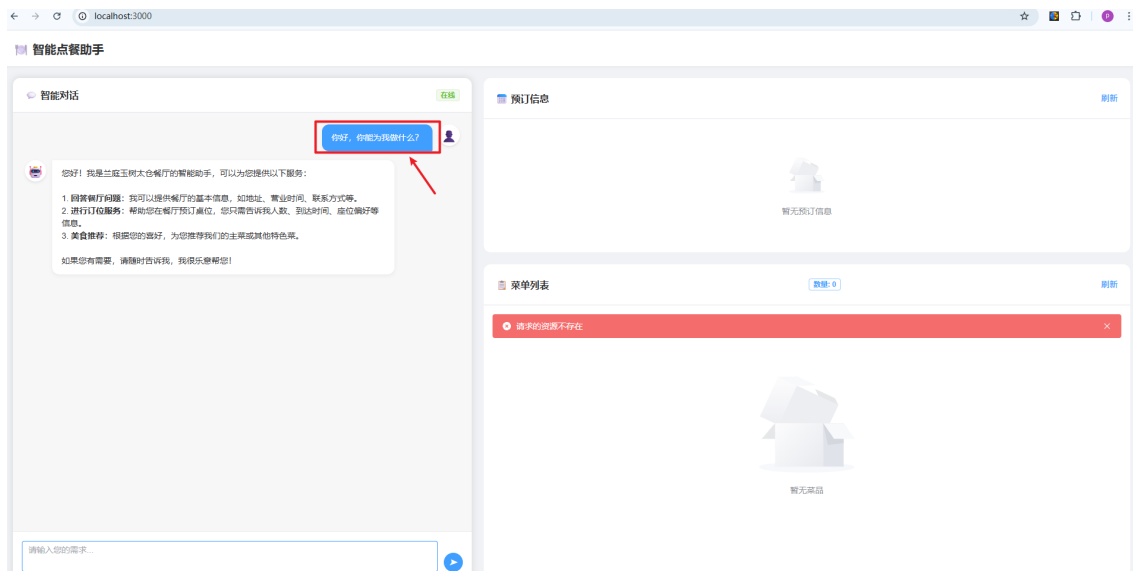
```
1 npm run dev
```

提示：开发环境下，Vite会将 `/api` 路径代理到 `http://localhost:8000`，确保后端服务在该地址运行。

运行完成后，在地址栏输入：`http://localhost:3000`



接下来，可以通过页面实现聊天过程：`你好，你能为我做什么？`



测试工具调用情况：`你们这里有什么菜可以推荐吗？`

你们这里有什么菜可以推荐吗？



我们餐厅有以下主菜值得推荐，您可以根据个人喜好选择：

1. 宫保鸡丁

- 价格: ¥28.00
- 描述: 经典川菜，鸡肉丁配花生米，酸甜微辣，口感丰富。
- 辣度等级: 🌶️🌶️
- 主要食材: 鸡胸肉、花生米、青椒、红椒、葱段。
- 过敏信息: 含花生，可能有麸质。

2. 清炒时蔬

- 价格: ¥15.00
- 描述: 新鲜时令蔬菜清炒，营养健康，口感清爽。
- 辣度等级: 无辣度
- 主要食材: 时令蔬菜、蒜蓉。
- 适合素食者。

3. 红烧鲈鱼

- 价格: ¥45.00
- 描述: 新鲜鲈鱼红烧制作，肉质鲜美，营养丰富。
- 辣度等级: 🌶️
- 主要食材: 鲈鱼、生抽、老抽、冰糖、葱姜。
- 过敏信息: 含鱼类。

您有兴趣尝试其中的哪一道菜呢？或者，想要了解更多种类的菜品？

我想吃豆腐类的菜品，有什么推荐吗？

我想吃豆腐类的菜品，有什么推荐吗？



我们餐厅有一道经典的豆腐菜品推荐给您：

麻婆豆腐

- 价格: ¥18.00
- 描述: 四川传统名菜，嫩滑豆腐配麻辣汤汁，是下饭神器！
- 辣度等级: 🌶️🌶️🌶️
- 口味: 麻辣鲜香
- 主要食材: 嫩豆腐、牛肉末、豆瓣酱、花椒。
- 过敏信息: 含大豆，可能含有麸质。

这道菜麻辣鲜香，非常适合喜欢重口味的朋友。您是否想尝试这道菜呢？

你们这里可以接受预定吗？

你们这里可以接受预定吗？



当然可以！我们餐厅提供预订服务，您只需要提供以下信息，我就可以为您安排：

1. 人数（包括儿童人数，若有的话）。

2. 到达时间。

3. 座位偏好（比如靠窗、安静区域等）。

4. 主菜偏好（可选）。

5. 其他备注需求（如是否需要特殊服务）。

请告诉我您的需求，我会帮助您完成预订！

请帮我预定一下，明天晚上8点的靠窗的位置，2个人，不带小朋友的，主菜偏辣一点的



这个时候我们去数据库中查询看看是否有预定的记录：

```
mysql> use menu
Database changed
mysql> select * from reservation_order;
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | num_people | num_children | arrival_time | seat_preference | main_dish_preference | other_comments | created_at | updated_at |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 4 | 1 | 2023-08-01 18:00:00 | 靠窗 | 牛排 | 希望有个好位置 | 2026-04-13 22:30:03 | 2026-04-13 22:30:03 |
| 2 | 2 | 0 | 2026-04-19 20:00:00 | 靠窗 | 偏辣 | | 2026-04-18 16:02:13 | 2026-04-18 16:02:13 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> |
```

四、FAQ

例如，本项目中可以问 **你们餐厅位置在哪里？**，那么像这种问题如果也交给大模型处理就完全没必要了，因为这种问题，可以认为是一个静态、固定的答案，也就是 **FAQ** 了。

FAQ 是 **Frequently Asked Questions** 的缩写，中文译为“**常见问题解答**”。

FAQ 是一份整理好的文档或页面，集中列出某个主题、产品、服务或领域中用户最常提出的问题，并提供对应的标准答案。

主要作用

作用	说明
☒ 提升效率	用户可快速自助查找答案，减少重复咨询
☒ 降低客服压力	减少人工客服处理基础问题的负担
☒ 改善用户体验	新用户能快速上手，老用户可快速定位问题
☒ 统一信息口径	避免因人工回复不一致造成的误解

具体实现流程

该接口在用户输入问题过程当中调用，每当用户输入停顿时，即调用该接口匹配 **FAQ** 库当中的相关问题：

您可能想问:

位置在哪里?

位置

具体实现逻辑如下:

- 获取用户问题;
- 查找当前库中所有 FAQ 问题;
- 计算用户问题和库中所有 FAQ 问题的相似度评分;
- 按照特定阈值进行筛选, 得到最相似的 n 个问题;
- 回复相关问题和答案;

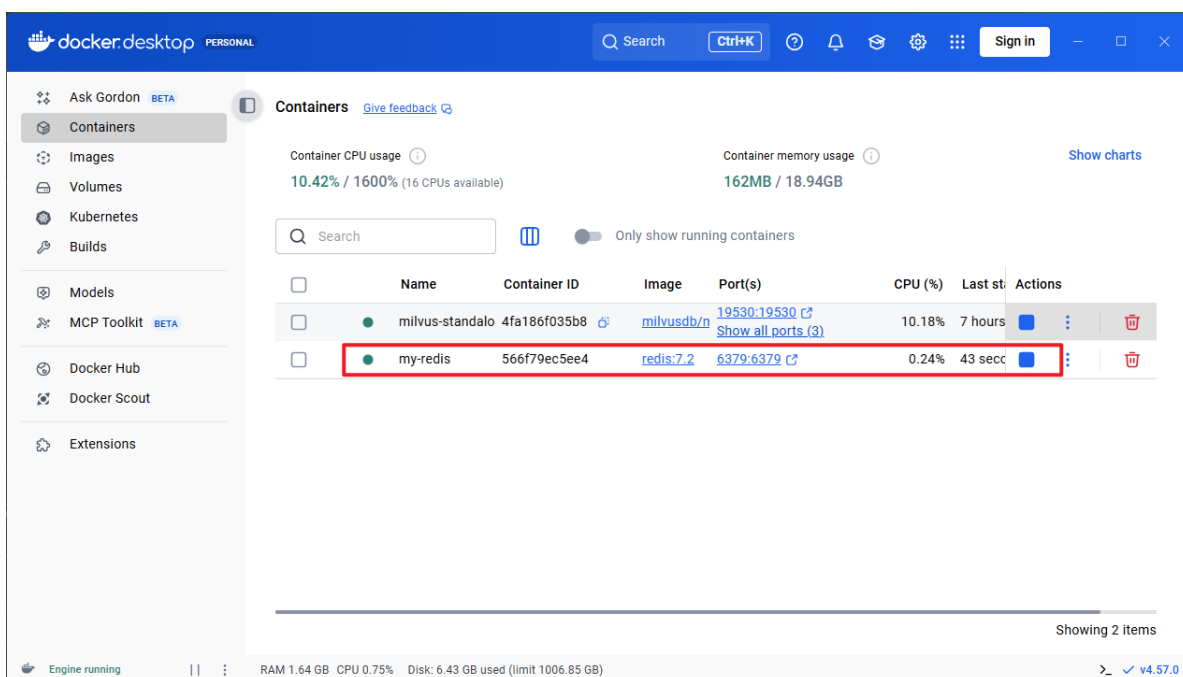
用户点击相关问题时, 直接回复答案, 无需调用 /chat 接口。

本项目需通过 Redis 来实现 faq 功能。在项目资料中已经提供 redis:7.2 的 tar 包文件, 首先将该文件通过如下命令导入到 docker 当中:

```
1 docker load -i redis.tar
```

通过如下命令即可将docker运行起来:

```
1 docker run -d --name my-redis -p 6379:6379 redis:7.2
```



4.1 Redis基本操作

在agent目录中创建 redis_demo.py 文件, 具体内容如下:

```
1 from redis import Redis
2 def redis_command_demo1():
3     """
4     redis相关的命令demo演示
5     """
6     # 1、获取到client对象
7     client = Redis.from_url("redis://localhost:6379", decode_responses=True)
```

```

8
9      # 2、使用client对象，执行一些命令：
10      # 命令很多，可以将命令做一些分类：
11      # 字符串相关的命令
12      # hash map相关的命令
13      # list相关的命令
14      # set相关的命令
15      # zset相关的命令
16
17      # 2.1 字符串相关的命令：执行set命令，创建一个value类型为string的key-value对
18      client.set("name", "张三")
19
20      # 执行get命令，获取到key为name的value
21      name = client.get("name")
22      print(name)
23
24      # 2.2 hash map 相关的命令：创建一个value为hash map 的 key-value对
25      client.hset(
26          "faq:items:address:test",
27          mapping={
28              "question": "地址是多少？ ",
29              "answer": "上海杨浦区"
30          }
31      )
32
33      # 获取到某一个key所对应hash map
34      faq_item = client.hgetall("faq:items:address:test")
35      print(faq_item)
36
37      # # 2.3 set相关的命令： value为一个集合的key-value对
38      # 添加了一个 key=faq:items value={"address", "phone", "email"} 的一个key-value对
39      client.sadd(
40          "faq:items3",
41          "address", "phone", "email"
42      )
43
44      # result就是key=faq:items 对应的value集合
45      result = client.smembers(
46          "faq:items3"
47      )
48      print(result)
49
50
51 def redis_command_demo2():
52     """
53     redis的pipeline命令的演示：
54     1、
55     """
56     # 1、获取到client对象
57     client = Redis.from_url("redis://localhost:6379", decode_responses=True)
58
59     # 2、通过client，获取到pipeline对象
60     pipeline = client.pipeline()
61
62     # 3、使用pipeline声明，需要执行的命令
63     pipeline.set("name3", "张三")
64     import time
65     time.sleep(30)

```

```

66     pipeline.get("name2")
67
68     pipeline.hset(
69         "faq:items:phone:test",
70         mapping={
71             "question": "手机号是多少? ",
72             "answer": "13800000000"
73         }
74     )
75
76     # 4、 执行pipeline中的命令
77     results = pipeline.execute()
78     print(results)
79
80     redis_command_demo1()

```

4.2 FAQ 数据同步至 Redis 中

构造 FAQ 数值对，并且将其通过批处理脚本，同步至Redis当中去，在 `agent` 目录下创建文件

`redis_data_sync.py`，示例代码如下：

```

1  import asyncio
2  import json
3  from typing import Dict, List, Optional, Tuple
4  from redis.asyncio import Redis
5  from pathlib import Path
6  import sys
7
8  ROOT_PATH = Path(__file__).parent.parent
9  sys.path.insert(0, str(ROOT_PATH))
10
11  from configs import config
12
13  FAQ_ITEMS: List[Dict[str, str]] = [
14      {"id": "business_hours", "question": "营业时间是什么时候?", "answer": "我们的营业时间是每天早9点至晚10点。"},
15      {"id": "address", "question": "位置在哪里?", "answer": "我们位于上海人民广场。"},
16      {"id": "phone", "question": "联系方式是什么?", "answer": "您可以添加官方微信shanghai_123, 或拨打电话 021-11111111。"},
17      {"id": "parking", "question": "停车方便吗?", "answer": "可优先使用附近公共停车场或路边划线车位; 车位实时情况建议电话确认: 021-11111111。"},
18      {"id": "promotion", "question": "有什么优惠活动?", "answer": "优惠活动: 单次消费满200九折, 满500八折。"},
19  ]
20
21  async def _get_redis_client() -> Redis:
22      cfg = config.REDIS_CONFIG
23      return Redis(
24          host=cfg['host'], port=cfg['port'], db=cfg['db'],
25          password=cfg['password'], decode_responses=True # 建议改为 True 可避免手动
decode
26      )
27
28  async def sync_faq_to_redis(
29      namespace: str = "faq",
30      items: Optional[List[Dict[str, str]]] = None,
31      replace: bool = False,

```

```

32 ) -> Tuple[int, str]:
33     r = await _get_redis_client()
34     faq_items = items or FAQ_ITEMS
35     ids_key = f"{namespace}:ids"
36     meta_key = f"{namespace}:meta"
37
38     # 2. 实现 replace 逻辑: 安全清理旧数据
39     if replace:
40         old_ids = await r.smembers(ids_key)
41         if old_ids:
42             # decode_responses=False 时返回 bytes, 需转为字符串
43             keys_to_del = [f"{namespace}:item:{key}" for key in old_ids]
44             if keys_to_del:
45                 await r.delete(*keys_to_del)
46             await r.delete(ids_key, meta_key)
47
48     pipe = r.pipeline()
49     for item in faq_items:
50         faq_id = item.get("id")
51         question = item.get("question")
52         answer = item.get("answer")
53         if not all([faq_id, question, answer]):
54             continue
55         key = f"{namespace}:item:{faq_id}"
56         pipe.hset(key, mapping={"id": faq_id, "question": question, "answer":
answer})
57         pipe.sadd(ids_key, faq_id)
58
59     pipe.hset(meta_key, mapping={"count": str(len(faq_items))})
60     res = await pipe.execute()
61
62     written = sum(1 for x in res if isinstance(x, int) and x > 0)
63     return written, ids_key
64
65 def export_faq_json(namespace: str = "faq", items: Optional[List[Dict[str, str]]]
= None) -> str:
66     payload = {"namespace": namespace, "items": items or FAQ_ITEMS}
67     return json.dumps(payload, ensure_ascii=False)
68
69 async def import_faq_json(
70     json_str: str,
71     redis_client: Optional[Redis] = None,
72     replace: bool = True,
73 ) -> Tuple[int, str]:
74     obj = json.loads(json_str)
75     namespace = obj.get("namespace") or "faq"
76     raw_items = obj.get("items") or []
77
78     safe_items = [
79         {"id": str(x["id"]), "question": str(x["question"]), "answer":
str(x["answer"])}
80         for x in raw_items
81         if isinstance(x, dict) and all(k in x for k in ("id", "question",
"answer"))
82     ]
83     return await sync_faq_to_redis(redis_client=redis_client,
namespace=namespace, items=safe_items, replace=replace)
84

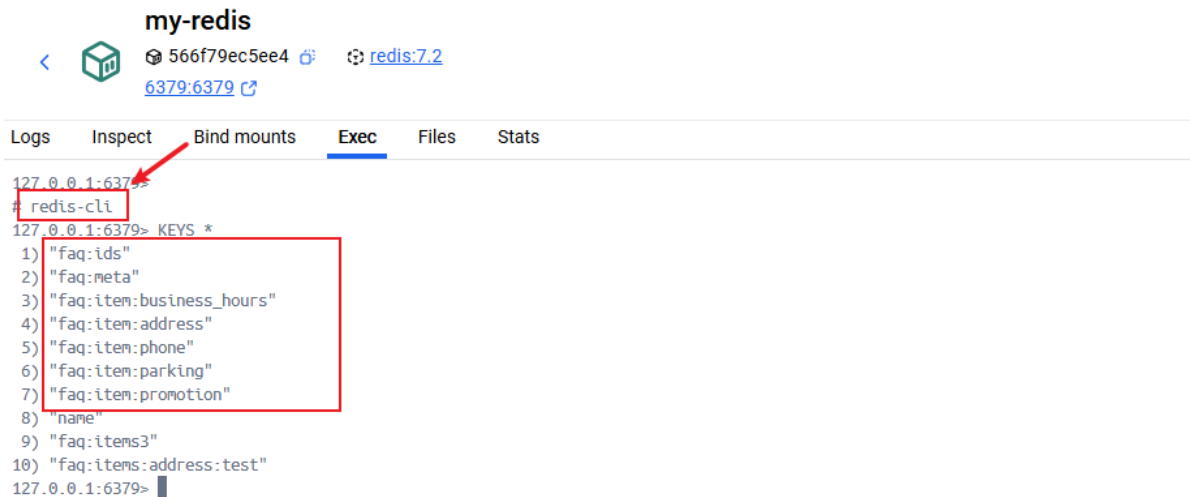
```

```

85     async def main():
86         # 4. 正确 await 异步调用
87         written, ids_key = await sync_faq_to_redis(
88             namespace=config.REDIS_CONFIG.get('faq_redis_namespace', 'faq'),
89             replace=True
90         )
91         print("sync_faq_to_redis:", written, ids_key)
92
93     if __name__ == "__main__":
94         asyncio.run(main())

```

运行后，进入 redis 容器，通过 KEYS * 命令查看具体生成的 keys 如下：



```

my-redis
566f79ec5ee4 redis:7.2
6379:6379

Logs Inspect Bind mounts Exec Files Stats
127.0.0.1:6379> # redis-cli
127.0.0.1:6379> KEYS *
1) "faq:ids"
2) "faq:meta"
3) "faq:item:business_hours"
4) "faq:item:address"
5) "faq:item:phone"
6) "faq:item:parking"
7) "faq:item:promotion"
8) "name"
9) "faq:items3"
10) "faq:items:address:test"
127.0.0.1:6379>

```

4.2 FAQ 数据加载

数据写入至 Redis 后，在项目中，就需要定义一个方法，用以将 FAQ 数据加载到内存中，后面需要用于和用户输入问题进行数据比对。在 main.py 文件中定义加载函数如下所示：

```

1     from pathlib import Path
2     import sys
3     ROOT_PATH = Path(__file__).parent.parent
4     sys.path.insert(0, str(ROOT_PATH))
5
6     from redis.asyncio import Redis
7     from typing import List
8     from pydantic import BaseModel
9
10    from configs import config
11
12    #其他代码省略
13
14    _redis_client = Redis(
15        host=config.REDIS_CONFIG['host'], port=config.REDIS_CONFIG['port'],
16        db=config.REDIS_CONFIG['db'],
17        password=config.REDIS_CONFIG['password'], decode_responses=True # 建议改为
18        True 可避免手动 decode
19    )
20
21    class FaqItem(BaseModel):
22        id:str
23        question:str
24        answer:str

```

```

23  async def _load_faq_items_from_redis(namespace: str = "faq") -> List[FaqItem]:
24      """
25      从redis加载faq问题
26      :param namespace: Description
27      :type namespace: str
28      :return: Description
29      :rtype: List[FaqItem]
30      """
31      # 1、构建client 对象
32      redis_client = await _redis_client
33      pipeline = redis_client.pipeline()
34
35      # 2、从redis set 中获取到faq所对应的所有keys
36      faq_ids = await redis_client.smembers(f"{namespace}:ids")
37
38      # 3、加载faq_keys所对应的所有faq，每个faq，构造成FaqItem对象
39      full_keys = [f"{namespace}:item:{fid}" for fid in faq_ids]
40      for faq_key in full_keys:
41          pipeline.hgetall(faq_key) #注意: Pipeline 的命令不需要 await，直接调用即可
42
43      all_faq_items = await pipeline.execute()
44      return [
45          FaqItem(
46              id=item["id"],
47              question=item["question"],
48              answer=item["answer"]
49          )
50          for item in all_faq_items
51      ]
52
53  async def main():
54      # 1、加载faq数据到redis中
55      res = await _load_faq_items_from_redis()
56      print("当前加载的faq数据为: ", res)
57  if __name__ == "__main__":
58      import asyncio
59      asyncio.run(main())

```

运行结果如下：

```

1  当前加载的faq数据为: [FaqItem(id='promotion', question='有什么优惠活动?', answer='优惠活动: 单次消费满200九折, 满500八折。'), FaqItem(id='address', question='位置在哪里?', answer='我们位于上海人民广场。'), FaqItem(id='phone', question='联系方式是什么?', answer='您可以添加官方微信 shanghai_123, 或拨打电话 021-11111111。'), FaqItem(id='parking', question='停车方便吗?', answer='可优先使用附近公共停车场或路边划线车位; 车位实时情况建议电话确认: 021-11111111。'), FaqItem(id='business_hours', question='营业时间是时候?', answer='我们的营业时间是每天早9点至晚10点。')]

```

4.3 计算用户问题和 FAQ 问题相似度得分

对于用户问题和 FAQ 问题，直接通过字符串之间相似度来进行相似度判断得分。当前得分函数用了三种策略：

- `SequenceMatcher.ratio()`（字符序列相似）：基于最长公共子序列（LCS）等，衡量字符级别的相似程度；
- `Jaccard`（字符集合相似）：使用两个字符串构成的集合的交集比上两个字符串当中字符构成的并集；

- 包含关系 **bonus** : 当一个字符串, 完全包含另外一个字符串时, 额外添加一些bonus

在 `main.py` 文件中继续添加如下代码：

```

1 from difflib import SequenceMatcher
2 import re
3
4 # 定义正则表达式，用于移除文本中的标点符号
5 _PUNCT_RE = re.compile(r"[\s,, .! ! ? ; : \\'\"'() \[\] \{\} <> 《》 、 /\\|_ -]+")
6
7 # 对文本进行清洗的函数：
8 def _normalize_text(text: str) -> str:
9     return _PUNCT_RE.sub("", (text or "").strip())
10
11 def _get_similarity_score(query:str, faq_question:str)->float:
12     """
13     使用简单的字符串匹配算法，计算query和faq_question的相似度得分
14     （备注：实际生产环境下，可以通过更复杂的算法，比如进行embedding再来计算余弦相似度等，来计算
15     相似度得分）
16     """
17     # 1、算法一：使用一个包：difflib.SequenceMatcher
18     # 底层原理：递归地比较两个字符串的最长公共子序列
19     # 例如： a = abcdef b = rtabcedgef
20     # 第一步：找到这两个字符串里面最长的公共子序列：abc
21     # 第二步：把a串切成三部分["", "abc", "def"] 把b串同样也切成三部分：
22     ["rt", "abc", "edgef"]
23     # 第三步：分别比较a串和b串左侧和右侧的部分，再找出公共最长子序列，依此类推，直到找
24     不到最长公共子序列
25     # 第四步：计算得分：2*所有最长公共子序列的长度和 / (a的长度+b的长度)
26     # 通过该算法，能够捕捉到query和faq_question之间的带位置信息的相似度
27     # 该算法的问题：
28     # 举例：query：密码忘记了怎么办，question：忘记了密码怎么办
29     # 针对于这种基本包含的关键词一致，但是关键词的语序位置不一样的匹配，得分会偏低
30     q = _normalize_text(query)
31     c = _normalize_text(faq_question)
32
33     sequence_matcher = SequenceMatcher(None, q, c)
34     score = sequence_matcher.ratio()
35
36     # 2、算法二：实现一个算法，只计算query包含的关键词/字和question包含的关键词/字，所构成一个词袋，之间的相似度。
37     # 此处引入Jaccard相似度：Jaccard相似度用来计算两个集合之间的相似度
38     # Jaccard相似度的定义： set a 和 set b 的交集的元素数量 / (set a 和 set b 的并集的数量)
39     a = set(list(q))
40     b = set(list(c))
41     jaccard_score = len(a.intersection(b)) / len(a.union(b))
42
43     bonus = 0.0
44     if q in c or c in q:
45         bonus = 0.2
46
47     # 3、对这两个分数做一个加权
48     return 0.6*score + 0.4*jaccard_score + bonus

```

4.4 FAQ 接口封装

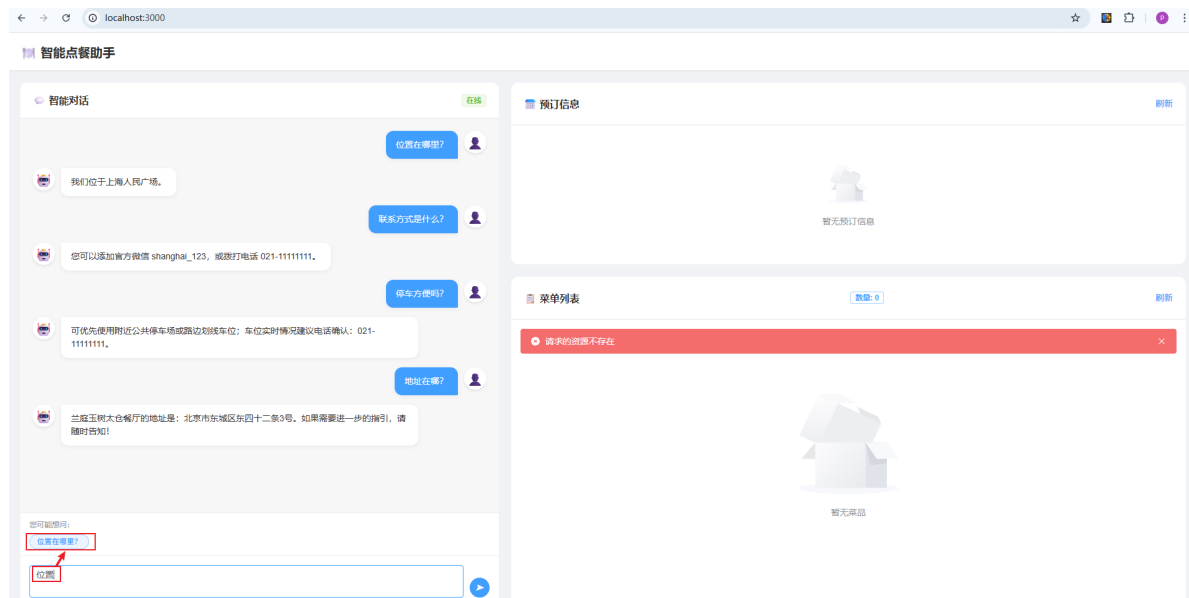
具体实现逻辑如下：

- 获取用户问题；
- 查找当前库中所有 FAQ 问题；
- 计算用户问题和库中所有 FAQ 问题的相似度评分；
- 按照特定阈值进行筛选，得到最相似的 n 个问题；
- 回复相关问题和答案；

在 `main.py` 中实现 FAQ 接口如下：

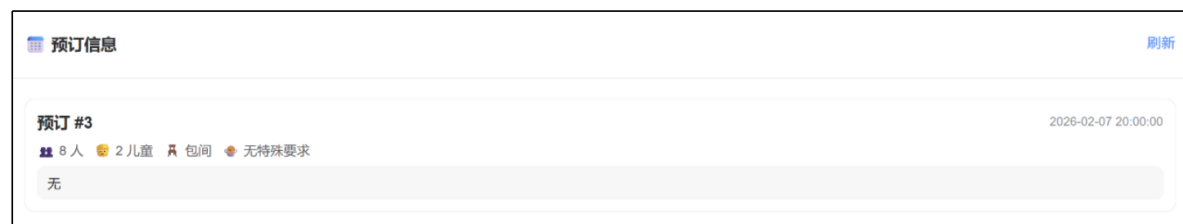
```
1 class FAQResponse(BaseModel):
2     success:bool
3     query:str
4     # 针对于用户的一个query，需要给到前端多个faq，
5     suggestions:list[FaqItem]
6
7 @app.get("/faq/suggest",response_model=FAQResponse)
8 async def faq_endpoint(query:str, limit:int=1):
9     limit = 1
10    # 1、从redis中获取所有的faq 的数据
11    faq_items = await _load_faq_items_from_redis()
12    # 2、将这些数据中的question和用户的query，进行比较，得到相似度得分
13    scores_list = []
14    for faq_item in faq_items:
15        score = _get_similarity_score(query, faq_item.question)
16        scores_list.append((score, faq_item))
17        # 此处可以根据实际情况，调整不同的阈值，来筛选出哪些question是和用户query相关的
18        # 如果不配置阈值的，总能够得到一个结果
19    # 3、将这些得分进行排序，得到最相似的前1条数据
20    scores_list.sort(key=lambda x:x[0],reverse=True)
21
22    top_k_items = scores_list[:limit]
23    # 4、将这个结果，返回给前端
24    return FAQResponse(
25        success=True,
26        query=query,
27        suggestions=[faq_item for score, faq_item in top_k_items]
28    )
```

启动前端服务，运行如下：



五、预订单查询

该接口用于查询所有预订单，效果如下所示：



直接通过查询MySQL数据即可，代码如下：

```
1  from typing import Optional, List
2  from agent.langchain_assistant import mysql_connection
3  from datetime import datetime
4  from sqlalchemy import text
5
6
7  ...
8
9  class ReservationItem(BaseModel):
10     id: int
11     num_people: int
12     num_children: int
13     arrival_time: Optional[str] = None
14     seat_preference: Optional[str] = None
15     main_dish_preference: Optional[str] = None
16     other_comments: Optional[str] = None
17     created_at: Optional[str] = None
18
19  class ReservationListResponse(BaseModel):
20     success: bool
21     reservations: List[ReservationItem]
22     count: int
23     message: str
24
25  @app.get("/reservation/list", response_model=ReservationListResponse)
```

```

26  async def reservation_list_endpoint(limit: int = 1):
27      """
28      获取到预订列表
29      """
30      # 获取到sqlalchemy.engine.Connection, 这个对象和pymysql.Connection有点区别
31      with mysql_connection().connect() as conn:
32          sql = """
33          select
34              id,
35              num_people,
36              num_children,
37              arrival_time,
38              seat_preference,
39              main_dish_preference,
40              other_comments,
41              created_at
42          from
43              menu.reservation_order
44          """
45          # 直接通过conn.execute().fetchall() 获取到的是一个列表, 列表的每个元素是一个元组
46          # results = conn.execute(text(sql)).fetchall()
47          # 通过conn.execute().mappings().fetchall() 获取到的是一个列表, 列表的每个元素是
          一个字典
48          results = conn.execute(text(sql)).mappings().fetchall()
49          # 把results 转换为 ReservationItem 模型的列表
50          item_list = []
51          for result in results:
52              item = ReservationItem(
53                  id=result["id"],
54                  num_people=result["num_people"],
55                  num_children=result["num_children"],
56                  arrival_time=datetime.strptime(result["arrival_time"], "%Y-%m-%d
%H:%M:%S"),
57                  seat_preference=result["seat_preference"],
58                  main_dish_preference=result["main_dish_preference"],
59                  other_comments=result["other_comments"],
60                  created_at=datetime.strptime(result["created_at"], "%Y-%m-%d
%H:%M:%S"),
61              )
62              item_list.append(item)
63
64          return ReservationListResponse(
65              success=True,
66              reservations=item_list,
67              count=len(item_list),
68              message="success"
69          )

```

六、菜品查询

该接口用以查询所有菜品, 效果如下:



直接通过 MySQL 查询数据即可，代码如下：

```
1 class MenuItem(BaseModel):
2     id:int
3     dish_name:str
4     price:float
5     formatted_price:str # 格式化后的价格，例如：¥12.50
6     description:str
7     category:str
8     spice_level:int
9     spice_text:str
10    flavor:str
11    main_ingredients:str
12    cooking_method:str
13    is_vegetarian:bool
14    allergens:str
15    is_available:bool
16
17 # 定义菜品列表响应模型
18 class MenuListResponse(BaseModel):
19     """菜品列表响应"""
20     success: bool
21     menu_items: List[MenuItem] # 菜品列表
22     count: int # 菜品数
23     message: str # 响应消息提示
24
25 @app.get("/menu/list", response_model=MenuListResponse)
26 async def menu_list():
27     """
28     获取到菜单列表
29     """
30     with mysql_connection().connect() as conn:
31         sql = """
32             SELECT
33
34                 id, dish_name, price, description, category,
35                 spice_level, flavor, main_ingredients,
36                 is_vegetarian, allergens, is_available
37             FROM menu_items
38             WHERE is_available = 1
39             ORDER BY category, dish_name
40         """
41         results = conn.execute(text(sql)).mappings().fetchall()
42         # 把results 转换为 MenuItem 模型的列表
43         item_list = []
44
45         for result in results:
```

```
45     spice_levels = {0: "不辣", 1: "微辣", 2: "中辣", 3: "重辣"}
46     spice_text = spice_levels.get(result["spice_level"], "未知")
47     item = MenuItem(
48         id=result["id"],
49         dish_name=result["dish_name"],
50         price=result["price"],
51         formatted_price=f"¥{result['price']:.2f}",
52         description=result["description"],
53         category=result["category"],
54         spice_level=result["spice_level"],
55         spice_text=spice_text,
56         flavor=result["flavor"],
57         main_ingredients=result["main_ingredients"],
58         cooking_method=result["cooking_method"],
59         is_vegetarian=result["is_vegetarian"],
60         allergens=result["allergens"],
61         is_available=result["is_available"],
62     )
63     item_list.append(item)
64
65     return MenuListResponse(
66         success=True,
67         menu_items=item_list,
68         count=len(item_list),
69         message="success"
70     )
```